STAT340 - Topic 5: Prediction

Simple linear regression (SLR) review

Quick overview of SLR (one predictor, one response) from 240:

- Let X, Y be numeric RVs with **clear linear relationship**; recall theoretical correlation:

$$\rho_{X,Y} = \frac{\mathrm{Cov}(X, Y)}{\sqrt{\mathrm{Var}[X]\,\mathrm{Var}[Y]}}$$

  In sample, we observe **sample correlation coefficient**:

$$r = \frac{\frac{1}{n-1}\sum(x_i - \bar{x})(y_i - \bar{y})}{s_x s_y}$$

Simple linear regression (SLR) review

Quick overview of SLR (one predictor, one response) from 240:

- Let X, Y be numeric RVs with **clear linear relationship**; recall theoretical correlation:

$$\rho_{X,Y} = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}[X]\,\text{Var}[Y]}}$$

  In sample, we observe **sample correlation coefficient**:

$$r = \frac{\frac{1}{n-1}\sum(x_i - \bar{x})(y_i - \bar{y})}{s_x s_y}$$

- Note $n-1$ used in numerator instead of n, similar to sample SDs s_x, s_y
- ρ, r always in $[-1, 1]$, where:
  - Sign (positive/negative/zero) indicates **direction of slope** of best fit line, and
  - Absolute value indicates **strength of correlation** (closer to ±1 means stronger)
- **Note:** correlation coefficient ***assumes*** linear relationship; ***must*** be checked with plot!
- Interactive correlation demo: https://bwu62.shinyapps.io/correlation

SLR model

Let (x_i, y_i) be sample of size n of apparently linear points. **Model equation** for SLR:

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$$

- β_0, β_1 are true intercept & slope (treated as constants like x_i)
- $\varepsilon_i \overset{\text{iid}}{\sim} \text{Normal}(0, \sigma^2)$ are **errors** of each point (the only variable on RHS!)

SLR model

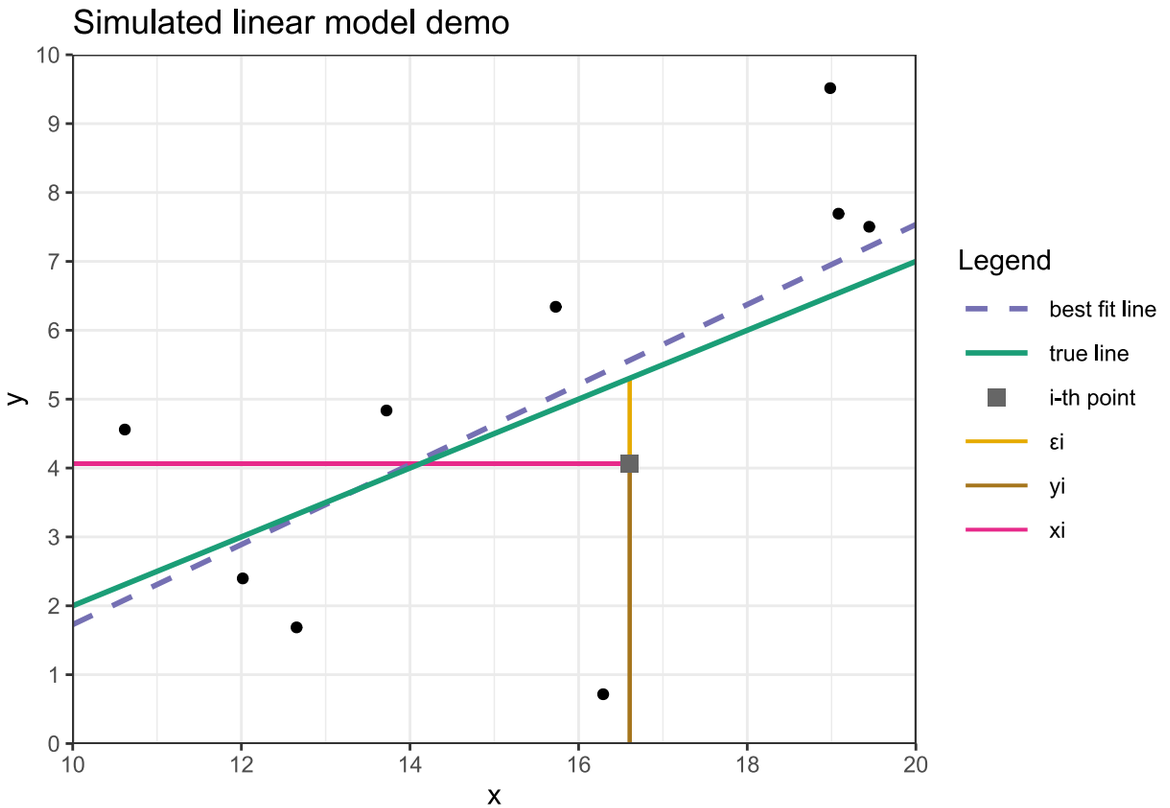
Let (x_i, y_i) be sample of size n of apparently linear points. **Model equation** for SLR:

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$$

- β_0, β_1 are true intercept & slope (treated as constants like x_i)
- $\varepsilon_i \overset{\text{iid}}{\sim} \text{Normal}(0, \sigma^2)$ are **errors** of each point (the only variable on RHS!)

MC simulated demo:

```
b0  = -3              # true intercept
b1  = 0.5             # true slope
n   = 10              # sample size
m   = 10              # min X range
M   = 20              # max X range
sig = 2               # true sigma
x   = runif(n,m,M)    # generate Xs
e   = rnorm(n,0,sig)  # generate errors
y   = b0 + b1*x + e   # combine for Ys
```

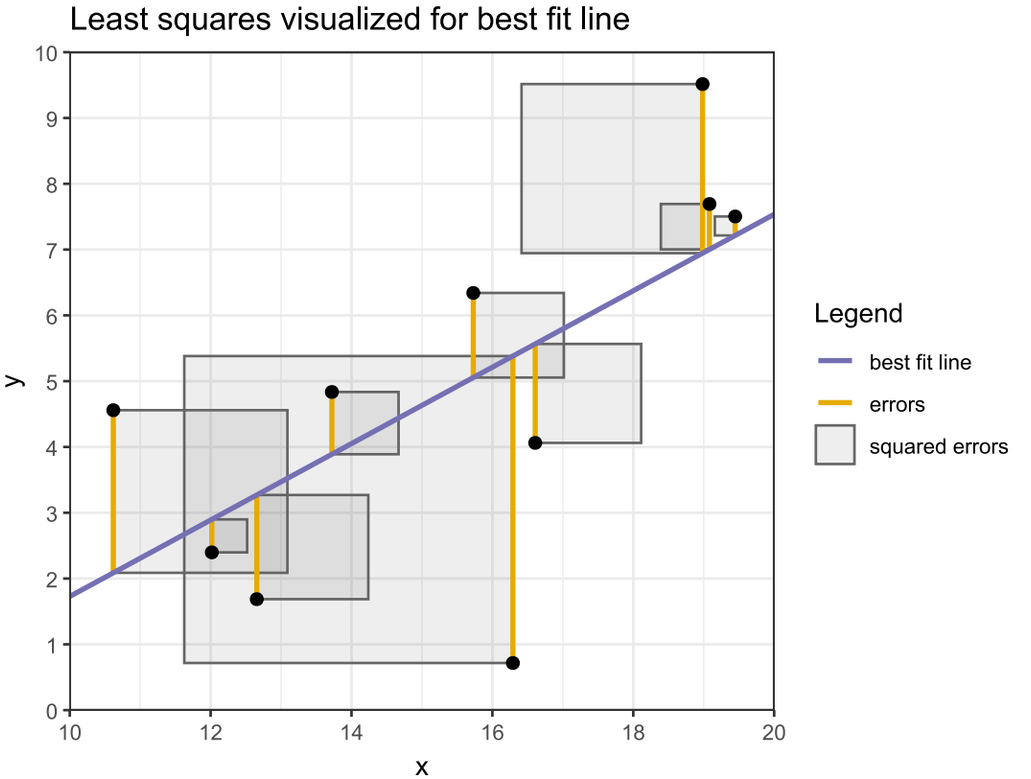


OLS fitting

Typically **ordinary least squares** (OLS) used to fit, i.e. **sum of squared errors** minimized. Given some β_0', β_1', define OLS **loss function**:

$$\ell(\beta_0', \beta_1') = \sum_{i=1}^{n} (y_i - (\beta_0' + \beta_1' x_i))^2$$

To minimize $\ell(\beta_0', \beta_1')$, set partial derivatives vs β_0', β_1' to 0 and solve (full derivation)

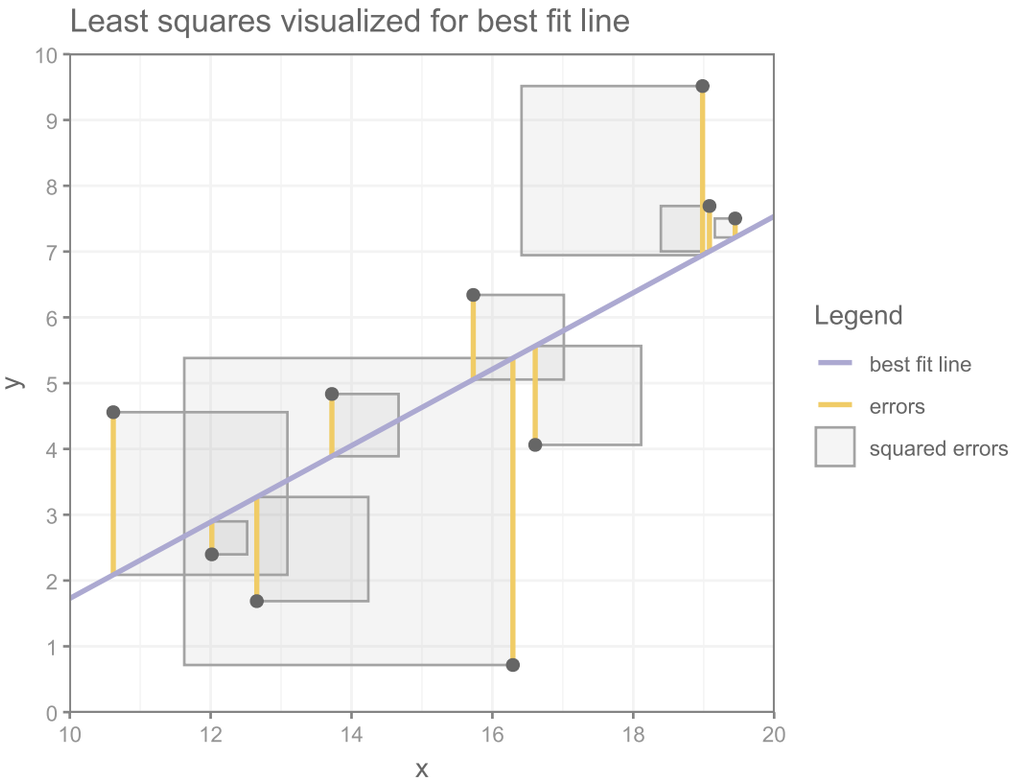


OLS fitting

Typically **ordinary least squares** (OLS) used to fit, i.e. **sum of squared errors** minimized. Given some β_0', β_1', define OLS **loss function**:

$$\ell(\beta_0', \beta_1') = \sum_{i=1}^{n} (y_i - (\beta_0' + \beta_1' x_i))^2$$

To minimize $\ell(\beta_0', \beta_1')$, set partial derivatives vs β_0', β_1' to 0 and solve (full derivation)



Best fit coefficients:

$$\begin{aligned} \hat{\beta}_1 &= r \cdot \frac{s_y}{s_x} \\ \hat{\beta}_0 &= \bar{y} - \hat{\beta}_1 \bar{x} \end{aligned}$$

- **Note:** equivalent to line with slope $\hat{\beta}_1$ passing through $(\bar{x}, \bar{y})$
- Can be shown these estimators are unbiased with least variance

Inference

After fitting, our **fitted** or **predicted values** $\hat{y}_i$ are found by pluggin x_i back:

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$$

Then our **residuals** $\hat{\varepsilon}_i$ are the realizations of $\varepsilon_i \sim \text{Normal}(0, \sigma^2)$, defined:

$$\hat{\varepsilon}_i = y_i - \hat{y}_i = y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_i)$$

Inference

After fitting, our **fitted** or **predicted values** $\hat{y}_i$ are found by pluggin x_i back:

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$$

Then our **residuals** $\hat{\varepsilon}_i$ are the realizations of $\varepsilon_i \sim \text{Normal}(0, \sigma^2)$, defined:

$$\hat{\varepsilon}_i = y_i - \hat{y}_i = y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_i)$$

Using these, we estimate σ^2 using the **mean squared error** of the residuals:

$$\hat{\sigma}^2 = \frac{\ell(\hat{\beta}_0, \hat{\beta}_1)}{n-2} = \frac{\sum \hat{\varepsilon}_i^2}{n-2} = \frac{1}{n-2} \sum_i (y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_i))^2$$

Can be shown $\hat{\sigma}^2$ is unbiased estimator for σ^2. Note: σ^2 is important since this represents the **amount of noise** in the data. Higher $\sigma^2 \rightarrow$ more noise $\rightarrow$ less precise results.

Remember to check if $\hat{\varepsilon}_i \sim \text{Normal}(0, \sigma^2)$ with residual plots!

Checking residuals

Recall $\varepsilon_i \overset{\text{iid}}{\sim} \text{Normal}(0, \sigma^2)$. The main assumptions we need to check with residuals are:

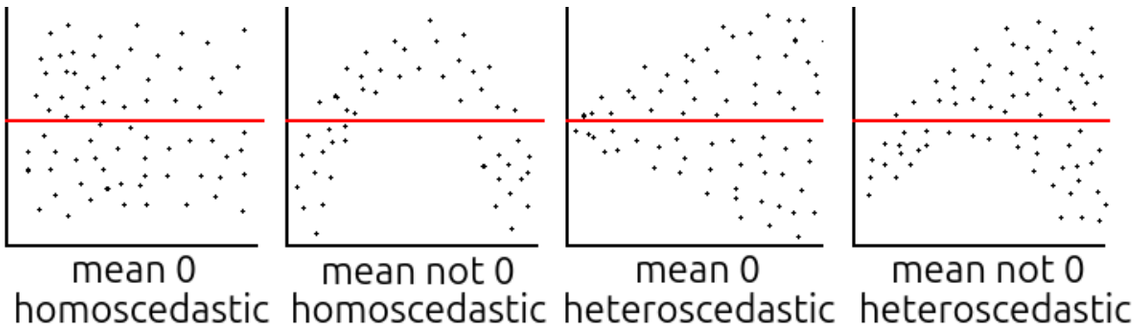
1. Residuals $\hat{\varepsilon}_i$ all centered around 0 across dataset (unbiased)
2. Residuals $\hat{\varepsilon}_i$ have constant variance across dataset (homoscedasticity)
3. Residuals $\hat{\varepsilon}_i$ collectively roughly normal

Checking residuals

Recall $\varepsilon_i \overset{\text{iid}}{\sim} \text{Normal}(0, \sigma^2)$. The main assumptions we need to check with residuals are:

1. Residuals $\hat{\varepsilon}_i$ all centered around 0 across dataset (unbiased)
2. Residuals $\hat{\varepsilon}_i$ have constant variance across dataset (homoscedasticity)
3. Residuals $\hat{\varepsilon}_i$ collectively roughly normal

For 1-2, plot $\hat{\varepsilon}_i$ vs $\hat{y}_i$ and check if points form flat band about y=0 horizontal line:

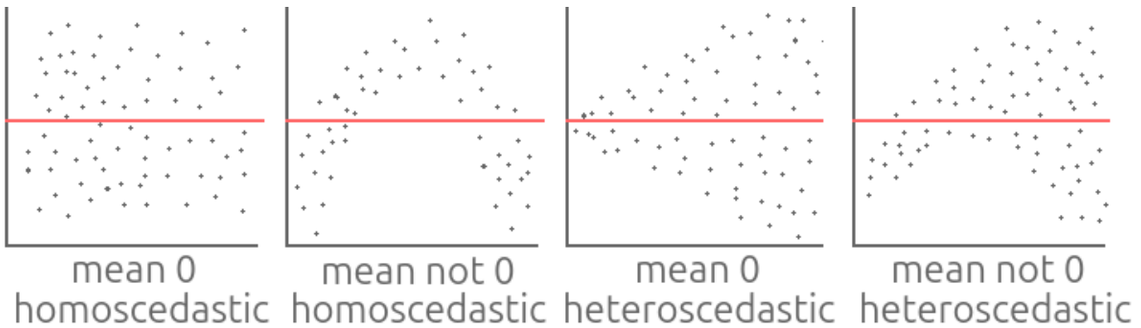


Checking residuals

Recall $\varepsilon_i \overset{\text{iid}}{\sim} \text{Normal}(0, \sigma^2)$. The main assumptions we need to check with residuals are:

1. Residuals $\hat{\varepsilon}_i$ all centered around 0 across dataset (unbiased)
2. Residuals $\hat{\varepsilon}_i$ have constant variance across dataset (homoscedasticity)
3. Residuals $\hat{\varepsilon}_i$ collectively roughly normal

For 1-2, plot $\hat{\varepsilon}_i$ vs $\hat{y}_i$ and check if points form flat band about y=0 horizontal line:



For 3, qq-plot the residuals (i.e. sample vs theoretical quantiles) and check for straight line:



SE of coefficients

Slope SE can be easily derived now. First, note:

$$\begin{aligned}\sum(x_i - \bar{x})(y_i - \bar{y}) &= \sum(x_i - \bar{x})y_i - \bar{y}\cancel{\sum(x_i - \bar{x})} \\ &= \sum(x_i - \bar{x})(\beta_0 + \beta_1 x_i + \varepsilon_i)\end{aligned}$$

SE of coefficients

Slope SE can be easily derived now. First, note:

$$\begin{aligned}\sum(x_i-\bar{x})(y_i-\bar{y}) &= \sum(x_i-\bar{x})y_i - \bar{y}\cancel{\sum(x_i-\bar{x})} \\ &= \sum(x_i-\bar{x})(\beta_0+\beta_1 x_i+\varepsilon_i)\end{aligned}$$

$$\begin{aligned}\mathrm{Var}[\hat{\beta}_1] &= \mathrm{Var}\left[\frac{\sum(x_i-\bar{x})(y_i-\bar{y})}{\sum(x_i-\bar{x})^2}\right] = \mathrm{Var}\left[\frac{\sum_i(x_i-\bar{x})(\cancel{\beta_0+\beta_1 x_i}+\varepsilon_i)}{\sum(x_i-\bar{x})^2}\right] \\ &= \frac{\sum(x_i-\bar{x})^2\mathrm{Var}(\varepsilon_i)}{\left(\sum(x_i-\bar{x})^2\right)^2} = \frac{\sigma^2}{\sum_i(x_i-\bar{x})^2}\end{aligned}$$

SE of coefficients

Slope SE can be easily derived now. First, note:

$$\begin{aligned}\sum (x_i - \bar{x})(y_i - \bar{y}) &= \sum (x_i - \bar{x}) y_i - \bar{y} \cancel{\sum (x_i - \bar{x})} \\ &= \sum (x_i - \bar{x})(\beta_0 + \beta_1 x_i + \varepsilon_i)\end{aligned}$$

$$\begin{aligned}\mathrm{Var}[\hat{\beta}_1] = \mathrm{Var}\left[\frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}\right] &= \mathrm{Var}\left[\frac{\sum_i (x_i - \bar{x})(\cancel{\beta_0 + \beta_1 x_i} + \varepsilon_i)}{\sum (x_i - \bar{x})^2}\right] \\ = \frac{\sum (x_i - \bar{x})^2 \mathrm{Var}(\varepsilon_i)}{\left(\sum (x_i - \bar{x})^2\right)^2} &= \frac{\sigma^2}{\sum_i (x_i - \bar{x})^2}\end{aligned}$$

Then, using our sample we can estimate $\mathrm{SE}(\hat{\beta}_1) = \sqrt{\dfrac{\hat{\sigma}^2}{s_x^2 (n-1)}}$. Similar process gives:

$\mathrm{Var}[\hat{\beta}_0] = \sigma^2 \left[\dfrac{1}{n} + \dfrac{\bar{x}^2}{\sum_i (x_i - \bar{x})^2}\right]$, so estimate $\mathrm{SE}(\hat{\beta}_0) = \sqrt{\hat{\sigma}^2 \left[\dfrac{1}{n} + \dfrac{\bar{x}^2}{s_x^2 (n-1)}\right]}$

Inference on coefficients

Coefficients $\hat{\beta}_0, \hat{\beta}_1$ follow T-distributions with $n-2$ degrees of freedom.

- For hypothesis testing, e.g. $H_0 : \beta_1 = 0$ vs $H_a : \beta_1 \neq 0$, use:

$$t_{\text{obs}} = \frac{\hat{\beta}_1 - 0}{\text{SE}(\hat{\beta}_1)} \sim T_{n-2}$$

Inference on coefficients

Coefficients $\hat{\beta}_0, \hat{\beta}_1$ follow T-distributions with $n-2$ degrees of freedom.

- For hypothesis testing, e.g. $H_0 : \beta_1 = 0$ vs $H_a : \beta_1 \neq 0$, use:

$$t_{\text{obs}} = \frac{\hat{\beta}_1 - 0}{\text{SE}(\hat{\beta}_1)} \sim T_{n-2}$$

- For confidence intervals, e.g. C% interval for β_1, use:

$$C\% \text{ or } (1-\alpha) \text{ interval} = \hat{\beta}_1 \pm t_{\alpha/2,\, n-2} \cdot \text{SE}(\hat{\beta}_1)$$

  - $t_{\alpha/2,\, n-2}$ is T critical value, e.g. `qt(0.975,df=n-2)` for 95%
  - Can also use `confint()` function

Confidence/prediction interval for predictions

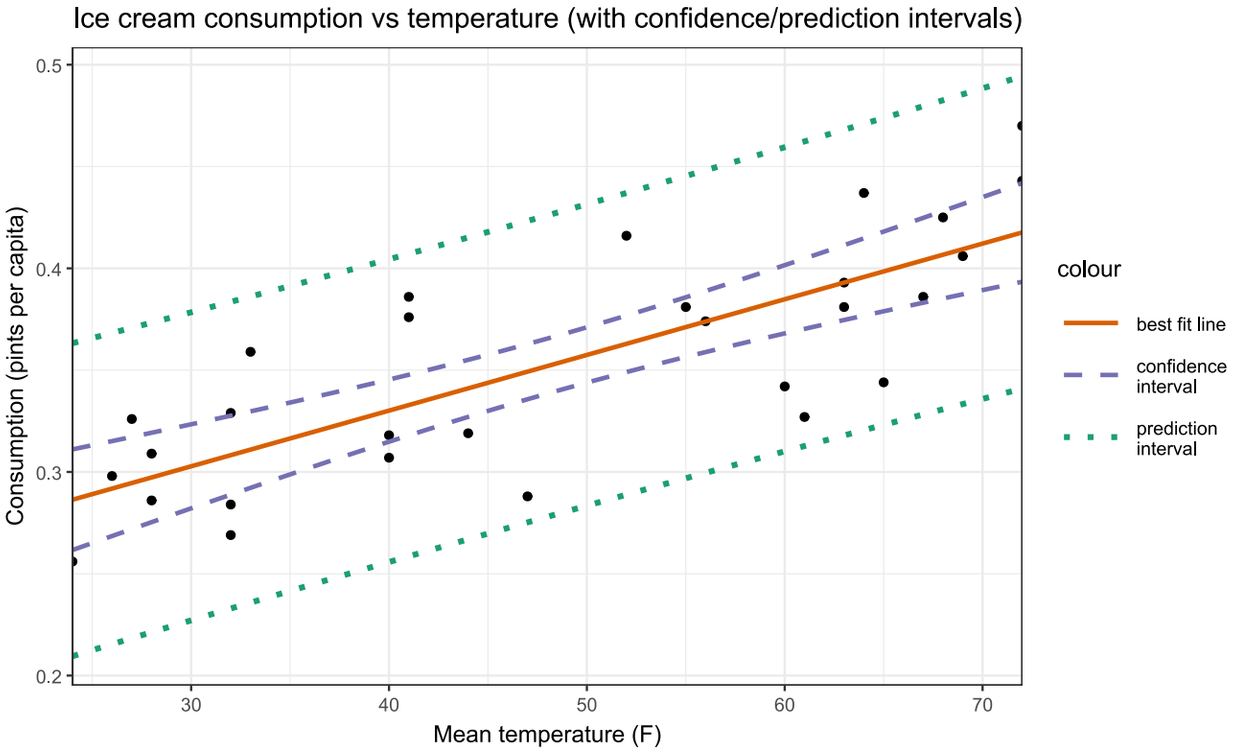
Suppose we take new observation at $X = x_{\text{new}}$, can make two different intervals at there:

- **Confidence interval** for mean predicted response $\hat{y}$ (i.e. for the regression line), which has:

$$\text{SE}(\hat{y}) = \sqrt{\hat{\sigma}^2 \left[\frac{1}{n} + \frac{(x_{\text{new}} - \bar{x})^2}{s_x^2 (n-1)} \right]}$$

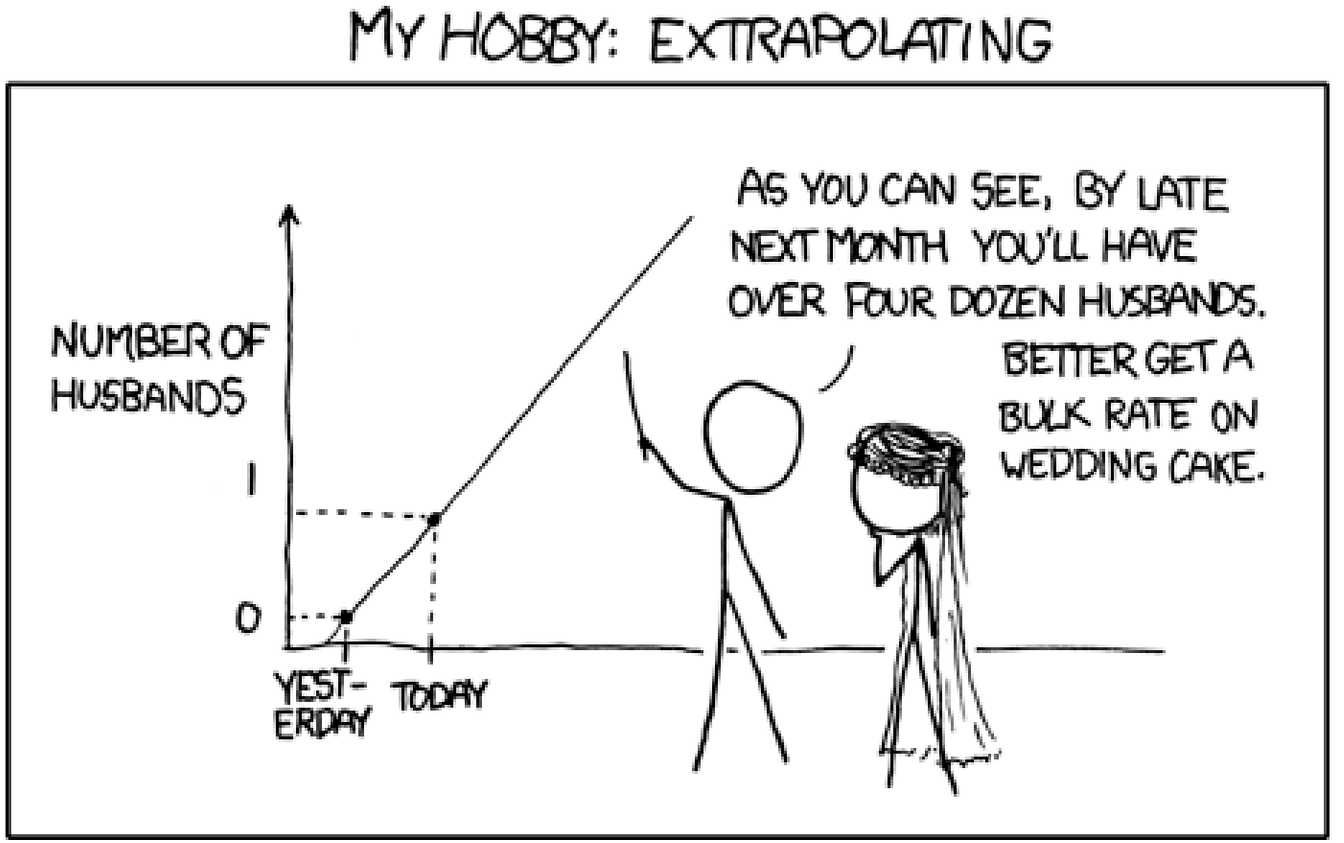
- **Prediction interval** for next individual data point y_{new} (i.e. for the data points), which has:

$$\text{SE}(y_{\text{new}}) = \sqrt{\hat{\sigma}^2 \left[1 + \frac{1}{n} + \frac{(x_{\text{new}} - \bar{x})^2}{s_x^2 (n-1)} \right]}$$



- Can also use `predict(lm.object, interval="confidence")` or `"prediction"`

Extrapolating



source: https://xkcd.com/605/

Cumulative SLR exercise!

Suppose you have the following observations:

```
x = c(14.4, 15.3, 11.2, 10.7, 11.9, 13, 18.4, 15.3, 17.3, 10.1, 18.4, 18.1, 16.6, 15.7, 10.3,
19, 10.9, 16.3, 17, 17.2, 14.3, 12.3, 15.8, 17.8, 15.2, 20, 11.1, 18.3, 15.9, 12.1)
y = c(50.9, 50.3, 41.7, 25, 34.6, 50.4, 65.5, 48.8, 62.2, 41, 64.4, 71.5, 66.8, 58.5, 38.8,
63.7, 36.5, 62.4, 68, 66.4, 49.7, 31.3, 51.3, 64.3, 56.3, 76.7, 38.8, 59.3, 55.1, 33.4)
```

Do the following (try both manually and using R functions):

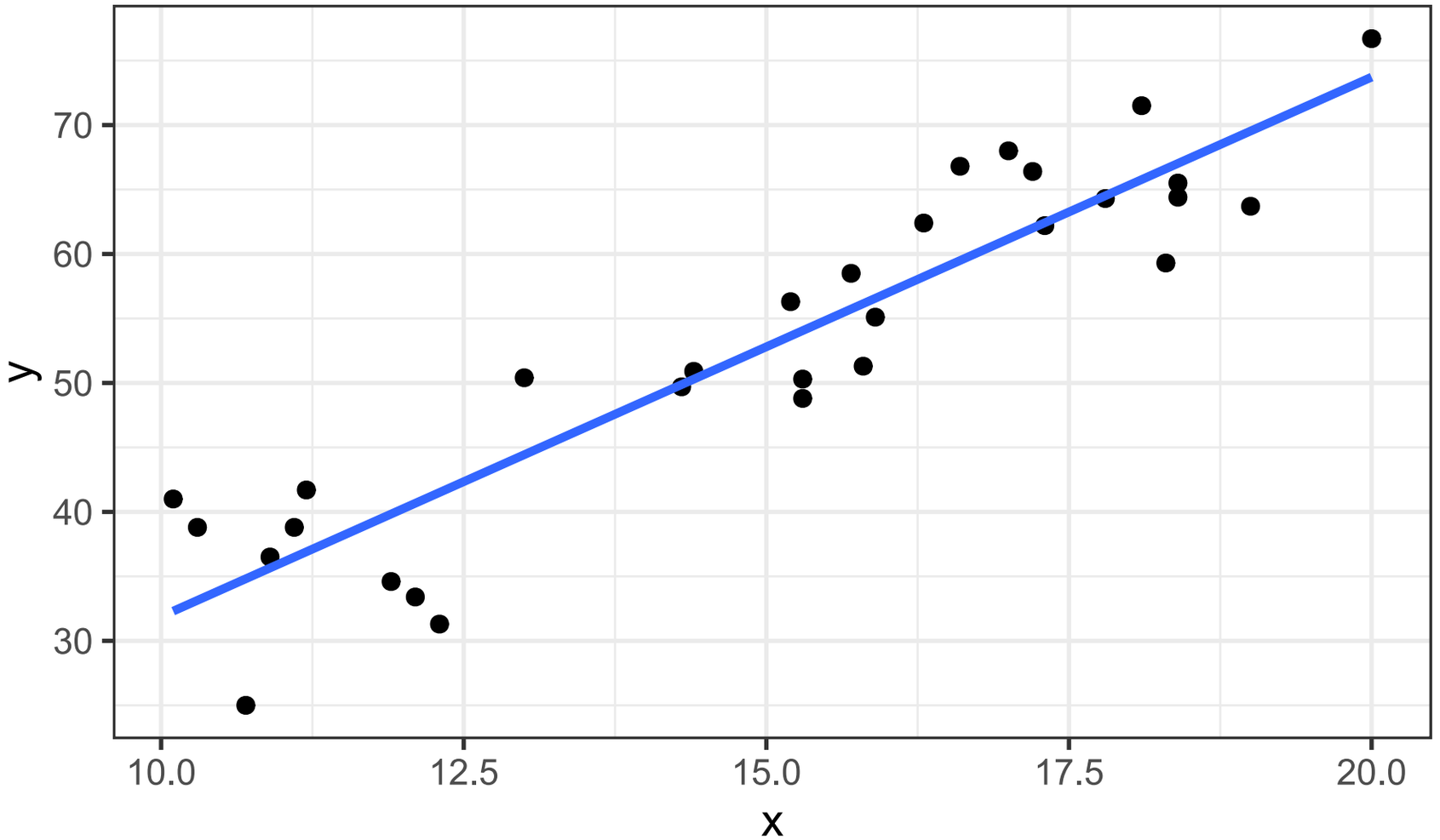
1. Plot the data, check if SLR is appropriate, and find the correlation r,
2. Fit a best fit line, plot it, and find the coefficients $\hat{\beta}_1, \hat{\beta}_0$,
3. Find the fitted values $\hat{y}_i$ and residuals $\hat{\varepsilon}_i$, and check model assumptions,
4. Estimate SD of errors σ and SEs of coefficients $\hat{\beta}_1, \hat{\beta}_0$,
5. Find p-value for testing $\beta_1 = 0$ vs not, and give 95% CIs for β_1, β_0,
6. At $X = 18$, give and interpret 95% confidence and prediction intervals.

Answers using R functions shown first:

Exercise answers using R functions

```r
# plot data, best fit line, and find correlation
ggplot(tibble(x,y),aes(x,y)) + geom_point() + geom_smooth(method=lm,se=F)
cor(x,y)
```

```
[1] 0.9196951
```



Exercise answers using R functions (cont.)

```r
# fit with lm(), print summary with coefficients, SEs, and residual SE
# also shows coefficient p-values for testing each =0 vs ≠0
model = lm(y ~ x, data = tibble(x,y))
summary(model)
```

```
Call:
lm(formula = y ~ x, data = tibble(x, y))

Residuals:
     Min       1Q   Median       3Q      Max
-10.2063  -4.5742   0.2184   4.3378   8.6965

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  -9.9455     5.1557  -1.929   0.0639 .
x             4.1831     0.3375  12.395 6.92e-13 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.381 on 28 degrees of freedom
Multiple R-squared:  0.8458,    Adjusted R-squared:  0.8403
F-statistic: 153.6 on 1 and 28 DF,  p-value: 6.916e-13
```

Exercise answers using R functions (cont.)

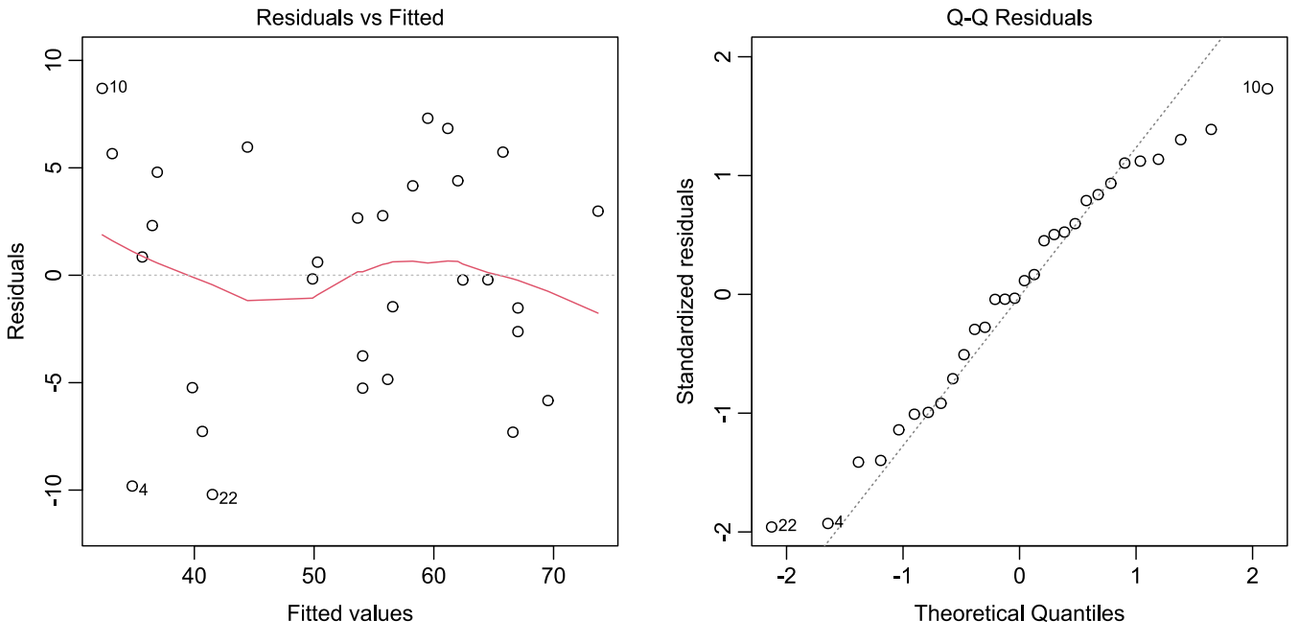
```
fitted(model) %>% unname %>% round(2)      # fitted values
```

```
 [1] 50.29 54.06 36.90 34.81 39.83 44.43 67.02 54.06 62.42 32.30 67.02 65.77 59.49 55.73 33.14 69.53
[17] 35.65 58.24 61.17 62.00 49.87 41.51 56.15 64.51 53.64 73.72 36.49 66.60 56.57 40.67
```

```
resid(model) %>% unname %>% round(2)       # residuals
```

```
 [1]   0.61  -3.76   4.80  -9.81  -5.23   5.97  -1.52  -5.26  -0.22   8.70  -2.62   5.73   7.31
[14]   2.77   5.66  -5.83   0.85   4.16   6.83   4.40  -0.17 -10.21  -4.85  -0.21   2.66   2.98
[27]   2.31  -7.30  -1.47  -7.27
```

```
par(mfrow=c(1,2)) ; plot(model,which=1:2)  # make side-by-side residual plots
```




```
# alternative manual way: plot(fitted(model),resid(model))
#                         plot(qnorm(ppoints(n)),sort(resid(model)))
```

Exercise answers using R functions (cont.)

```r
# confidence intervals for coefficients
confint(model, level = 0.95)
```

```
                2.5 %    97.5 %
(Intercept) -20.506486 0.6153899
x             3.491761 4.8743932
```

```r
# confidence/prediction intervals for prediction at new X=18
predict(model, newdata = data.frame(x=18), interval = "confidence")
```

```
       fit      lwr     upr
1 65.34984 62.45838 68.2413
```

```r
predict(model, newdata = data.frame(x=18), interval = "prediction")
```

```
       fit      lwr     upr
1 65.34984 53.95459 76.7451
```

Answers using manual computation shown on next slides:

Exercise answers computing manually

```r
# define useful values
n = length(x) ; mx = mean(x) ; my = mean(y) ; sx = sd(x) ; sy = sd(y) ; r = cor(x,y)
# best fit slope and intercept
c(b1 <- r*sy/sx, b0 <- my-b1*mx)
```

```
[1]  4.183077 -9.945548
```

```r
# fitted and residual values
fitted = b0 + b1*x  ;  resid = y - fitted
# residual standard error
(sigma.hat = sqrt(sum(resid^2)/(n-2)))
```

```
[1] 5.380917
```

```r
# estimated SE of coefficients
(se.b1 = sigma.hat*sqrt(1/(sx^2*(n-1))))
```

```
[1] 0.3374895
```

```r
(se.b0 = sigma.hat*sqrt(1/n+mx^2/(sx^2*(n-1))))
```

```
[1] 5.155683
```

```r
# test true slope =0 vs ≠0
(b1.stat = b1/se.b1)
```

```
[1] 12.39469
```

```r
(b1.pval = 2*pt(-abs(b1.stat), df=n-2))
```

```
[1] 6.916425e-13
```

Exercise answers computing manually (cont.)

```r
# confidence intervals for both coefficients
(t.crit = qt(0.975, df=n-2))
```

```
[1] 2.048407
```

```r
(b1 + c(-1,1)*t.crit*se.b1)
```

```
[1] 3.491761 4.874393
```

```r
(b0 + c(-1,1)*t.crit*se.b0)
```

```
[1] -20.5064861   0.6153899
```

```r
# confidence/prediction intervals for prediction at new X=18
(y.new = b0 + b1*18)
```

```
[1] 65.34984
```

```r
(se.conf = sigma.hat*sqrt(1/n+(18-mx)^2/(sx^2*(n-1))))
```

```
[1] 1.411564
```

```r
(se.pred = sigma.hat*sqrt(1+1/n+(18-mx)^2/(sx^2*(n-1))))
```

```
[1] 5.562983
```

```r
(y.new + c(-1,1)*t.crit*se.conf)
```

```
[1] 62.45838 68.24130
```

```r
(y.new + c(-1,1)*t.crit*se.pred)
```

```
[1] 53.95459 76.74510
```

Verify SEs with MC

Can verify the coefficient SEs with MC:

```
# use same X vector
x = c(14.4,15.3,11.2,10.7,11.9,13,18.4,15.3,17.3,10.1,18.4,18.1,16.6,15.7,10.3,
      19,10.9,16.3,17,17.2,14.3,12.3,15.8,17.8,15.2,20,11.1,18.3,15.9,12.1)
n = length(x)

# previous y points generated with:
b1 = 4 ; b0 = -8 ; sigma = 6

# generate many new samples, fit, and extract coefficients
betas = replicate(1e4, coef(lm(b0 + b1*x + rnorm(n,0,sigma) ~ x)) )
b0s = betas[1,] ; b1s = betas[2,]
c(mean(b0s), mean(b1s))
```

```
[1] -7.986806  3.998902
```

```
# empirical SEs
c(sd(b0s), sd(b1s))
```

```
[1] 5.7415962 0.3755352
```

```
# theoretical values
sigma * sqrt(1/n + mx^2/(sx^2*(n-1)))
```

```
[1] 5.748852
```

```
sigma * sqrt(1/(sx^2*(n-1)))
```

```
[1] 0.3763182
```

Bootstrap case-resampling for SLR

What if we have just one real dataset, can we still find SEs with MC? Yes! We can **resample n points** from our sample **with replacement**, called case-based **bootstrap**

```
x  # reusing same x, y from earlier
```

```
 [1] 14.4 15.3 11.2 10.7 11.9 13.0 18.4 15.3 17.3 10.1 18.4 18.1 16.6 15.7 10.3 19.0 10.9 16.3 17.0
[20] 17.2 14.3 12.3 15.8 17.8 15.2 20.0 11.1 18.3 15.9 12.1
```

```
y
```

```
 [1] 50.9 50.3 41.7 25.0 34.6 50.4 65.5 48.8 62.2 41.0 64.4 71.5 66.8 58.5 38.8 63.7 36.5 62.4 68.0
[20] 66.4 49.7 31.3 51.3 64.3 56.3 76.7 38.8 59.3 55.1 33.4
```

```
n = length(x)                           # sample size
betas = replicate(1e4, {                # setup bootstrap replicate loop
  indices = sample(n,replace=T)         # resample n indices with replacement
  coef(lm(y[indices] ~ x[indices]))     # get points, lm, get coefs
})                                      # coefs collected by replicate into wide matrix
b1s = betas[2,]  ;  b0s = betas[1,]     # extract coefs from rows of matrix for ease
sd(b1s)                                 # SE of best fit slopes
```

```
[1] 0.376565
```

```
quantile(b1s,c(.025,.975))              # 95% confidence interval for true slope
```

```
    2.5%    97.5%
3.500118 4.961745
```

Case-resample bootstrap is a powerful & versatile tool in nonparametric statistics

How "good" is fit?

Often how "good" a fit is can have 2 different interpretations:

- Is the model **appropriate** for the data? I.e. is the linear model assumption correct?
  - Answered by checking residual plots: are they normal, 0-mean, constant variance?

How "good" is fit?

Often how "good" a fit is can have 2 different interpretations:

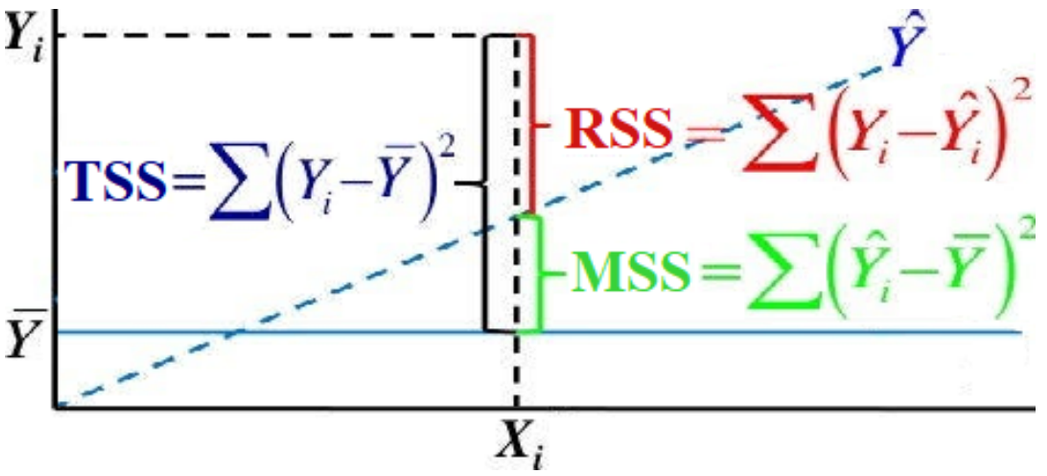
- Is the model **appropriate** for the data? I.e. is the linear model assumption correct?
  - Answered by checking residual plots: are they normal, 0-mean, constant variance?
- Alternatively, can ask how **predictive** is model? I.e. is X predicting Y well?
  - Often answered with **coefficient of determination** R^2, which $= (r)^2$ for SLR
  - Formal definition: recall data composed of "signal" and "noise" components; after fitting, **Total** variability of reponse, $\text{TSS} = \sum(y_i - \bar{y})^2$ (also SST) partitioned into:
    i. Portion explained by **Model** $\text{MSS} = \sum(\hat{y}_i - \bar{y})^2$ (also ESS or SSR)
    ii. **Residuals** portion not explained by model $\text{RSS} = \sum(y_i - \hat{y}_i)^2$ (also SSE or SSR)

How "good" is fit?

Often how "good" a fit is can have 2 different interpretations:

- Is the model **appropriate** for the data? I.e. is the linear model assumption correct?
  - Answered by checking residual plots: are they normal, 0-mean, constant variance?
- Alternatively, can ask how **predictive** is model? I.e. is X predicting Y well?
  - Often answered with **coefficient of determination** R^2, which $= (r)^2$ for SLR
  - Formal definition: recall data composed of "signal" and "noise" components; after fitting, **Total** variability of reponse, $\text{TSS} = \sum(y_i - \bar{y})^2$ (also SST) partitioned into:
    i. Portion explained by **Model** $\text{MSS} = \sum(\hat{y}_i - \bar{y})^2$ (also ESS or SSR)
    ii. **Residuals** portion not explained by model $\text{RSS} = \sum(y_i - \hat{y}_i)^2$ (also SSE or SSR)

$$R^2 = \frac{\text{TSS} - \text{RSS}}{\text{TSS}} = \frac{\text{MSS}}{\text{TSS}}$$
$$= 1 - \frac{\text{RSS}}{\text{TSS}}$$



R^2 example

For previous fit,

```
# print portion of summary table
summary(model)  %>% capture.output %>% tail(-8) %>% cat(sep="\n")
```

```
Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  -9.9455     5.1557  -1.929   0.0639 .
x             4.1831     0.3375  12.395 6.92e-13 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.381 on 28 degrees of freedom
Multiple R-squared:  0.8458,    Adjusted R-squared:  0.8403
F-statistic: 153.6 on 1 and 28 DF,  p-value: 6.916e-13
```

```
# compute R² from formula
1-sum(resid(model)^2)/(sy^2*(n-1))
```

```
[1] 0.845839
```

```
# compare with correlation²
r^2
```

```
[1] 0.845839
```

Interpretation: about 85% of variation in Y explained by variation in X

Multiple linear regression (MLR)

Generally speaking, MLR same as SLR but with more terms. Model equation:

$$y_i = \beta_0 + \beta_1 x_{i,1} + \beta_2 x_{i,2} + \cdots + \beta_p x_{i,p} + \varepsilon_i$$

- $\beta_0, \beta_1, ..., \beta_p$ are true intercept & slopes (still constants like all $x_{i,j}$)
- $\varepsilon_i \overset{\text{iid}}{\sim} \text{Normal}(0, \sigma^2)$ are **errors** of each point (still the only variable on RHS!)
- Still usually fit with OLS, i.e. make squares of distances from points as small as possible
- Derivation arguably simpler than SLR (details here)

Multiple linear regression (MLR)

Generally speaking, MLR same as SLR but with more terms. Model equation:

$$y_i = \beta_0 + \beta_1 x_{i,1} + \beta_2 x_{i,2} + \cdots + \beta_p x_{i,p} + \varepsilon_i$$

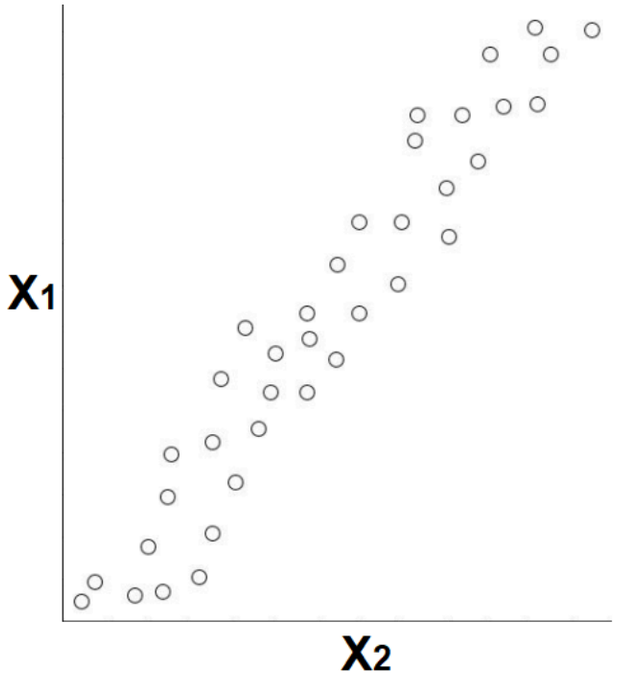
- $\beta_0, \beta_1, ..., \beta_p$ are true intercept & slopes (still constants like all $x_{i,j}$)
- $\varepsilon_i \overset{\text{iid}}{\sim} \text{Normal}(0, \sigma^2)$ are **errors** of each point (still the only variable on RHS!)
- Still usually fit with OLS, i.e. make squares of distances from points as small as possible
- Derivation arguably simpler than SLR (details here)

Only some key differences in MLR:

- Degrees of freedom: $n - (p + 1)$, one for each coefficient (note $p = 1$ gives SLR's $n - 2$)
- Coefficients interpreted as expected change in Y **while holding other variables constant**
  - MLR naturally attempts to attribute changes in Y associated with each variable
  - Useful to account/control for other influencing variables
  - Downside: predictors should not be strongly correlated, i.e. **multicollinearity** (can use statistics like VIF to detect)

Multicollinearity

To be clear, we want Y to be correlated with predictors, but we don't want predictors to be strongly correlated with each other, e.g:



This makes it harder to separate predictor effects and weakens MLR inference results. Can be resolved with variable selection (discussed later), special transformations, etc.

MLR exercise

Fitting in R: simply add more variables to `y ~ var1 + var2 + ...` formula in `lm()` call

Exercise: recall `mtcars` which contains fuel efficiency data for several cars, fit an MLR model predicting `qsec` using `disp`, `hp`, and `wt`. Show summary, interpret coefficients values + significances briefly, check residuals, and find 95% confidence intervals for coefficients.

MLR exercise

Fitting in R: simply add more variables to `y ~ var1 + var2 + ...` formula in `lm()` call

Exercise: recall `mtcars` which contains fuel efficiency data for several cars, fit an MLR model predicting `qsec` using `disp`, `hp`, and `wt`. Show summary, interpret coefficients values + significances briefly, check residuals, and find 95% confidence intervals for coefficients.

```
model.mtcars = lm(qsec ~ disp + hp + wt, data = mtcars)
summary(model.mtcars)  %>% capture.output %>% tail(-8) %>% cat(sep="\n")
```

```
Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 17.965050   0.849663  21.144  < 2e-16 ***
disp        -0.006622   0.004166  -1.590  0.12317
hp          -0.022953   0.004603  -4.986 2.88e-05 ***
wt           1.485283   0.429172   3.461  0.00175 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.062 on 28 degrees of freedom
Multiple R-squared:  0.6808,    Adjusted R-squared:  0.6466
F-statistic: 19.91 on 3 and 28 DF,  p-value: 4.134e-07
```

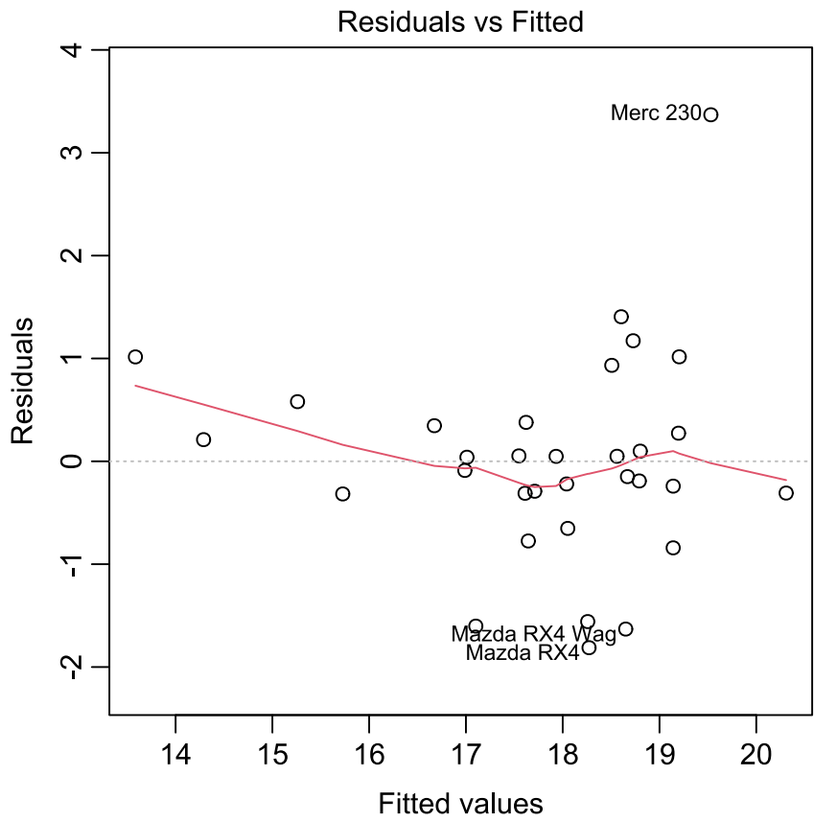
Interpretation, e.g. for `disp`: for each unit increase in displacement, qsec-time expected to drop by 0.0066 on average, **holding other variables constant**. Controlling for other variables, displacement not significant at $\alpha = 0.05$ ($p = 0.12$)

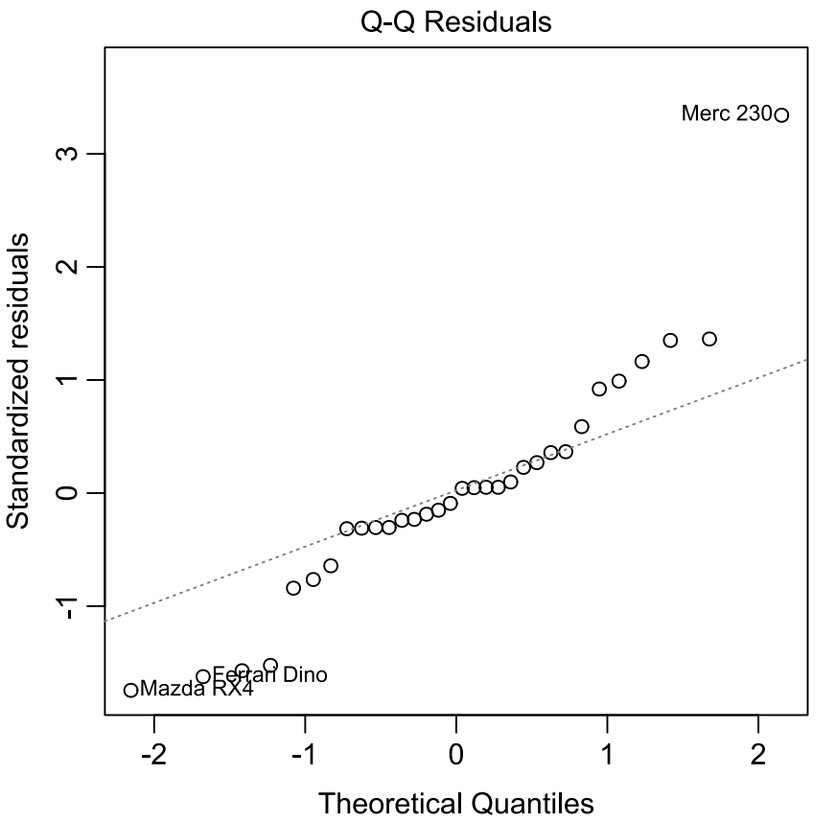
MLR exercise (cont.)

```
confint(model.mtcars)
```

```
                  2.5 %       97.5 %
(Intercept) 16.22459542 19.705504902
disp        -0.01515581  0.001911775
hp          -0.03238272 -0.013524148
wt           0.60616454  2.364401365
```

```
plot(model.mtcars, which=1:2)
```





MLR exercise (cont.)

For MLR, R^2 still defined as $1 - \mathrm{RSS}/\mathrm{TSS}$, but no longer interpretable as square of a correlation coefficient, since correlation only defined for two variables.

For previous exercise, compute and interpret R^2 using residuals.

MLR exercise (cont.)

For MLR, R^2 still defined as $1 - \mathrm{RSS}/\mathrm{TSS}$, but no longer interpretable as square of a correlation coefficient, since correlation only defined for two variables.

For previous exercise, compute and interpret R^2 using residuals.

```
# extract from summary output
summary(model.mtcars)$r.squared
```

```
[1] 0.6808292
```

```
# compute manually
1-sum(resid(model.mtcars)^2)/(var(mtcars$qsec)*(nrow(mtcars)-1))
```

```
[1] 0.6808292
```

Interpretation: about 68% of variation in qsec-time explained by variations in displacement, horsepower, and weight of cars.

Overall model F-test

Another way of assessing model significance: look at F-statistic, which is defined:

$$F_{\text{obs}} = \frac{\text{MSS}/p}{\text{RSS}/(n-(p+1))}$$

and tests against the null model in the null hypothesis:

$$H_0 : \beta_1 = \beta_2 = \cdots = \beta_p = 0,$$

vs H_a : at least one $\beta_i \neq 0$, i.e. tests entire model useless vs not useless.

Overall model F-test

Another way of assessing model significance: look at F-statistic, which is defined:

$$F_{\text{obs}} = \frac{\text{MSS}/p}{\text{RSS}/(n-(p+1))}$$

and tests against the null model in the null hypothesis:

$$H_0 : \beta_1 = \beta_2 = \cdots = \beta_p = 0,$$

vs H_a : at least one $\beta_i \neq 0$, i.e. tests entire model useless vs not useless.

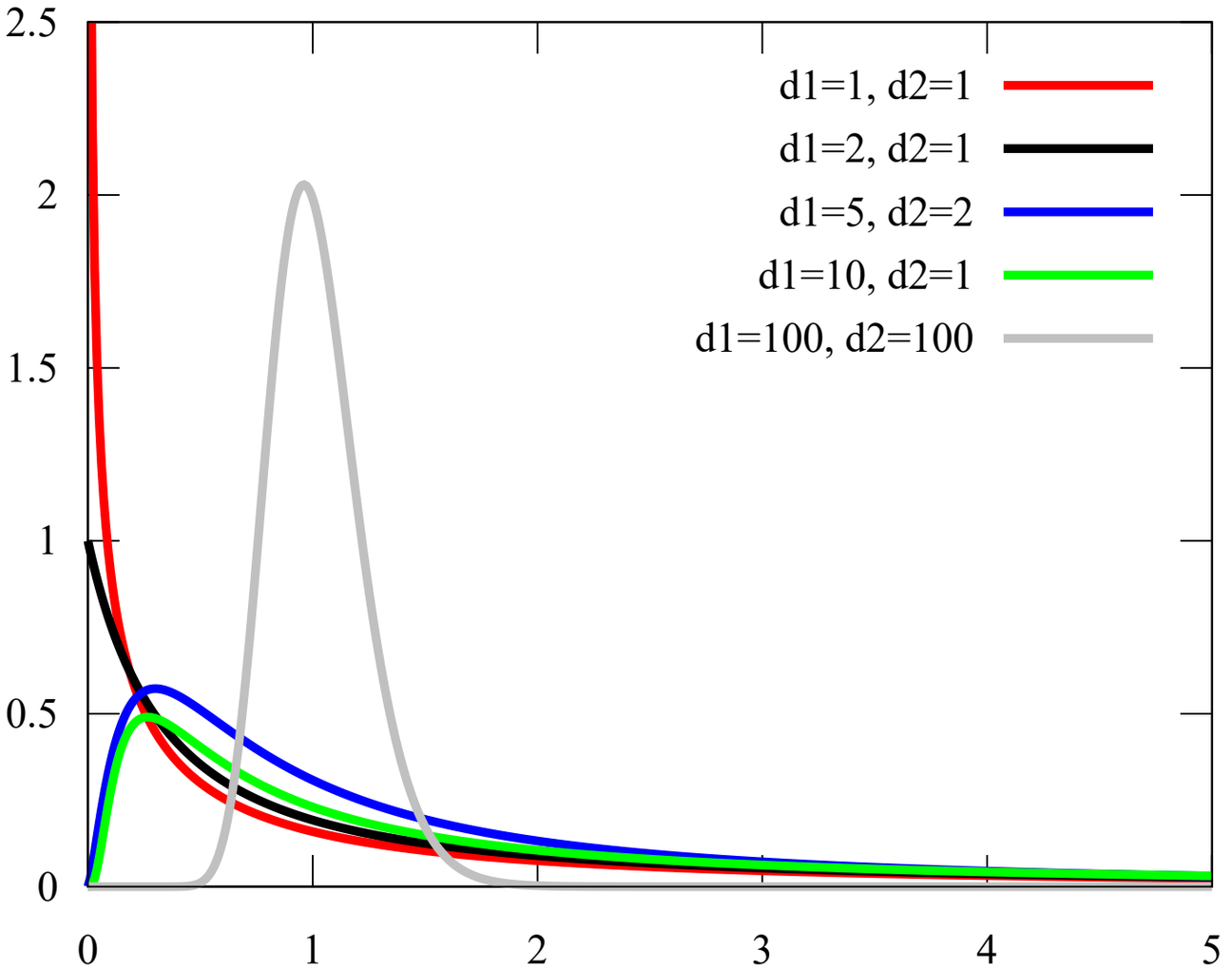
F-statistic follows F-distribution with $p, n-(p+1)$ DFs. For 340, just interpreting output of summary is sufficient:

```
summary(model.mtcars)  %>% capture.output %>% tail(4) %>% cat(sep="\n")
```

```
Residual standard error: 1.062 on 28 degrees of freedom
Multiple R-squared:  0.6808,    Adjusted R-squared:  0.6466
F-statistic: 19.91 on 3 and 28 DF,  p-value: 4.134e-07
```

$F = 19.91$ on 3, 28 DF, $p = 4.1 \times 10^{-7} \ll 0.05$, very significant vs null model

F-distribution examples



Additional predictor types

In MLR, there are often many other types of predictors we can consider adding to the model. We will examine 3 main types:

- Categorical variables
  - Examples: Control/treatment, male/female, high/medium/low, etc.
- Interactions
  - When 2 variables (numeric or categorical) have combined effects, i.e. “synergy”
- Non-linear terms
  - When a function of a variable is better modeled as linear vs response

Additional predictors can be added to model with same syntax `y ~ var1 + var2 + ...`

Categorical predictors

For categorical predictors with 2 or more **levels** (or categories), the following is done:

1. One level is **taken as the reference level**, (or baseline),
2. The difference between **every other level *vs the reference*** is then individually modeled as a **slope on an indicator variable** (i.e. 1/0) for that level,
3. These slopes are shown as coefficients (note: the **reference is NOT shown** as a slope!)

Categorical predictors

For categorical predictors with 2 or more **levels** (or categories), the following is done:

1. One level is **taken as the reference level**, (or baseline),
2. The difference between **every other level *vs the reference*** is then individually modeled as a **slope on an indicator variable** (i.e. 1/0) for that level,
3. These slopes are shown as coefficients (note: the **reference is NOT shown** as a slope!)

Example: for `iris` dataset below, `Species` is categorical with 3 levels:

```
str(iris)
```

```
'data.frame':   150 obs. of  5 variables:
 $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
 $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
 $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
 $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
 $ Species     : Factor w/ 3 levels "setosa","versicolor",..: 1 1 1 1 1 1 1 1 1 1 ...
```

```
# model predicting Petal.Length from all other variables:
# alternative: lm(Petal.Length ~ . , data=iris) where . means all other columns
model.iris = lm(Petal.Length ~ Sepal.Length + Sepal.Width + Petal.Width + Species, data=iris)
```

Iris example (cont.)

```
summary(model.iris)
```

```
Call:
lm(formula = Petal.Length ~ Sepal.Length + Sepal.Width + Petal.Width
+
    Species, data = iris)

Residuals:
     Min       1Q   Median       3Q      Max
-0.78396 -0.15708  0.00193  0.14730  0.65418

Coefficients:
                  Estimate Std. Error t value Pr(>|t|)
(Intercept)       -1.11099    0.26987  -4.117 6.45e-05 ***
Sepal.Length       0.60801    0.05024  12.101  < 2e-16 ***
Sepal.Width       -0.18052    0.08036  -2.246   0.0262 *
Petal.Width        0.60222    0.12144   4.959 1.97e-06 ***
Speciesversicolor  1.46337    0.17345   8.437 3.14e-14 ***
Speciesvirginica   1.97422    0.24480   8.065 2.60e-13 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.2627 on 144 degrees of freedom
Multiple R-squared:  0.9786,    Adjusted R-squared:  0.9778
F-statistic:  1317 on 5 and 144 DF,  p-value: < 2.2e-16
```

- Note: setosa used as reference (not shown), other species vs setosa fitted as coefficients
- E.g. **all else equal**, versicolor petal length higher vs setosa by 1.5cm **on average**
- Both species **significantly different** vs setosa (note: versicolor vs virginica difference NOT modeled & may not be significant)
- To change reference, mutate variable with `fct_relevel()`

  (*demo*)

Categorical variables (cont.)

Sometimes, categorical variables may already be coded as 1/0 indicator variables, in this case add them in as normal and the coefficient shown is the **difference of 1 vs 0 levels**. Example with `mtcars` again:

```
# update previous mtcars model by adding am, vs categorical variables
# alternative: model.mtcars2 = update(model.mtcars, . ~ . + am + vs)
model.mtcars2 = lm(qsec ~ disp + hp + wt + am + vs, data = mtcars)
summary(model.mtcars2)
```

```
Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 17.237081   1.083732  15.905 6.49e-15 ***
disp        -0.003235   0.003509  -0.922 0.364987
hp          -0.014683   0.003945  -3.722 0.000961 ***
wt           0.949956   0.358770   2.648 0.013584 *
am          -0.802919   0.462681  -1.735 0.094517 .
vs           1.787328   0.463080   3.860 0.000673 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.799 on 26 degrees of freedom
Multiple R-squared:  0.8323,    Adjusted R-squared:  0.8001
F-statistic: 25.81 on 5 and 26 DF,  p-value: 2.6e-09
```

I.e. **all else equal**, manual transmission faster vs automatic in qsec time by 0.8s **on average**, but this observed difference is only **weakly significant** ($p \approx 0.09$).

Practice exercise

```
str(penguins)
```

```
'data.frame':   344 obs. of  8 variables:
 $ species    : Factor w/ 3 levels "Adelie","Chinstrap",..: 1 1 1 1 1 1 1 1 1 1 ...
 $ island     : Factor w/ 3 levels "Biscoe","Dream",..: 3 3 3 3 3 3 3 3 3 3 ...
 $ bill_len   : num  39.1 39.5 40.3 NA 36.7 39.3 38.9 39.2 34.1 42 ...
 $ bill_dep   : num  18.7 17.4 18 NA 19.3 20.6 17.8 19.6 18.1 20.2 ...
 $ flipper_len: int  181 186 195 NA 193 190 181 195 193 190 ...
 $ body_mass  : int  3750 3800 3250 NA 3450 3650 3625 4675 3475 4250 ...
 $ sex        : Factor w/ 2 levels "female","male": 2 1 1 NA 1 2 1 2 NA NA ...
 $ year       : int  2007 2007 2007 2007 2007 2007 2007 2007 2007 2007 ...
```

With the build-in `penguins` dataset (shown above), do the following:

1. Use species, bill length, bill depth to linearly predict flipper length.
2. Give a full & clear interpretation of **each coefficient** (both slope & significance).
3. Is the overall model significant? Give a statistic & p-value.
4. What amount of the variation in flipper length is explained by your model?
5. Predict flipper length for an Adelie penguin with 40mm bill length and 20mm bill depth.
6. Run model diagnostics (i.e. check residual assumptions).

Penguins exercise answers

```r
model.penguins = lm(flipper_len ~ species + bill_len + bill_dep, data=penguins)
summary(model.penguins)
```

```
Coefficients:
                 Estimate Std. Error t value Pr(>|t|)
(Intercept)      127.6915     5.1937  24.586  < 2e-16 ***
speciesChinstrap  -1.3062     1.4391  -0.908    0.365
speciesGentoo     27.5648     1.9606  14.060  < 2e-16 ***
bill_len           0.7004     0.1197   5.851 1.16e-08 ***
bill_dep           1.9127     0.3162   6.050 3.85e-09 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.541 on 337 degrees of freedom
  (2 observations deleted due to missingness)
Multiple R-squared:  0.8465,    Adjusted R-squared:  0.8447
F-statistic: 464.7 on 4 and 337 DF,  p-value: < 2.2e-16
```

- Bill length found to be **highly significant** ($p \approx 10^{-8}$). **All else equal**, each 1mm increase in bill length associated with 0.70mm increase in flipper length **on average**.
- **All else equal**, Gentoo penguins' flipper lengths found to be **highly significantly** different vs Adelies ($p < 10^{-15}$) averaging 27.6mm longer, but Chinstraps not found to be significant ($p \approx 0.37$) averaging only -1.3mm shorter.

Penguins exercise answers (cont.)

```r
model.penguins = lm(flipper_len ~ species + bill_len + bill_dep, data=penguins)
summary(model.penguins)
```

```
Coefficients:
                 Estimate Std. Error t value Pr(>|t|)
(Intercept)      127.6915     5.1937  24.586  < 2e-16 ***
speciesChinstrap  -1.3062     1.4391  -0.908    0.365
speciesGentoo     27.5648     1.9606  14.060  < 2e-16 ***
bill_len           0.7004     0.1197   5.851 1.16e-08 ***
bill_dep           1.9127     0.3162   6.050 3.85e-09 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.541 on 337 degrees of freedom
  (2 observations deleted due to missingness)
Multiple R-squared:  0.8465,    Adjusted R-squared:  0.8447
F-statistic: 464.7 on 4 and 337 DF,  p-value: < 2.2e-16
```

- Overall model highly significant vs null ($F = 464.7$ on 4, 337 DFs, $p < 10^{-15}$).
- About 85% of variation in flipper length explained by model.

Penguins exercise answers (cont.)

```r
# prediction using predict() function
predict(model.penguins, tibble(species="Adelie",bill_len=40,bill_dep=20))
```

```
       1
193.9631
```

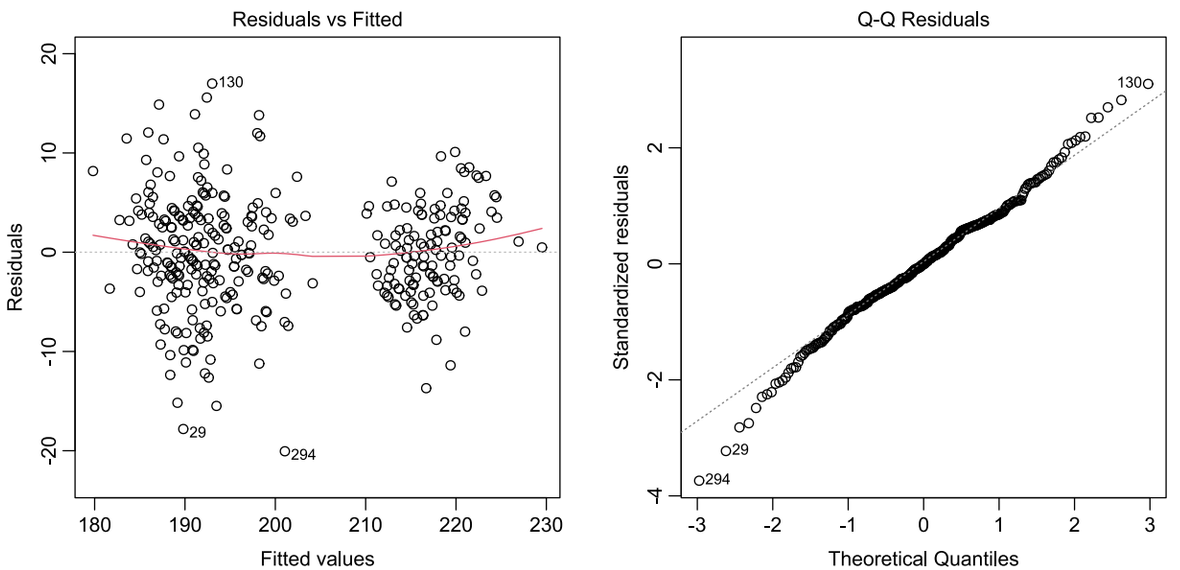
```r
# manual prediction
coef(model.penguins)
```

```
     (Intercept) speciesChinstrap    speciesGentoo         bill_len         bill_dep
     127.6914684       -1.3062319       27.5648475        0.7004416        1.9126995
```

```r
sum(coef(model.penguins) * c(1, 0, 0, 40, 20))
```

```
[1] 193.9631
```

```r
plot(model.penguins, which=1:2)
```



Categoricals in model equation

Mathematically, categorical variables add **indicator variable(s)** for each non-reference level. Suppose we predict Y with numeric X_1 and 3-level categorical X_2. Model equation:

$$y_i = \beta_0 + \beta_1 x_{i,1} + \Big(\beta_2 x_{i,2} + \beta_3 x_{i,3}\Big) + \varepsilon_i$$

Where β_2 is the "slope" for the 2nd level (vs 1st level as reference) and $x_{i,2}$ is the indicator variable which $= 1$ if a subject is in that group and $= 0$ otherwise.

Same thing applies for β_3 and $x_{i,3}$. If there are more levels, repeat for each.

- Note: each categorical predictor contributes $n - 1$ coefficients where n is # of levels

Interactions

Interaction effects are when **two (or more) predictors** have combined **synergistic effect**, i.e. there's a "pairing" effect where "the whole is more (or less) than the sum of the parts"

Mathematically, it's equivalent to adding a **slope for the *product*** of the variables.

Note: if a categorical variable is used in the interaction term, **each non-reference level** is **considered separately** in the product

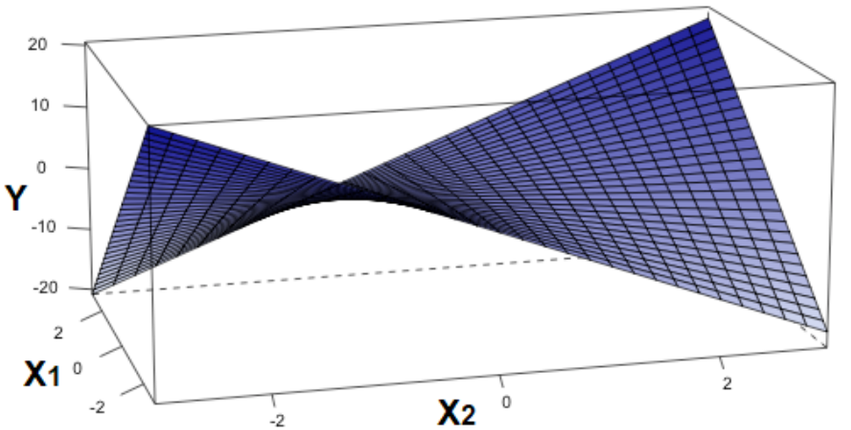
Interactions

Interaction effects are when **two (or more) predictors** have combined **synergistic effect**, i.e. there's a "pairing" effect where "the whole is more (or less) than the sum of the parts"

Mathematically, it's equivalent to adding a **slope for the *product*** of the variables.

Note: if a categorical variable is used in the interaction term, **each non-reference level** is **considered separately** in the product

Visually, it looks like a "twisting" of the regression plane, where the slope of X_i changes at a constant rate as X_j changes



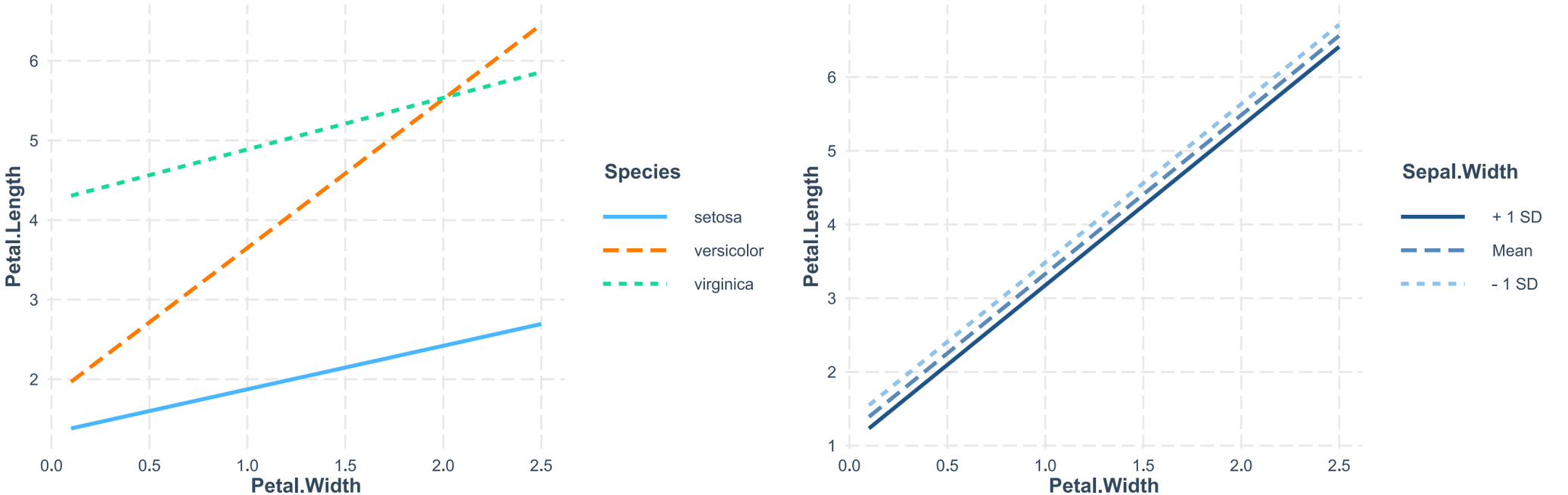
Examples of synergy effects:

- Pairing hot dogs with mustard, and ice cream with sprinkles
- Pairing alcohol with some drugs makes them more hazardous
- Combining different advertising mediums (e.g. TV / radio / social media) may significantly increase effectiveness of a marketing campaign

Interaction plot

How to detect if interaction exists? Make **interaction plot** where Y is plotted against both X_i and X_j and look for **changing slope** of X_i as X_j changes. Example with iris:

```
# can use any method e.g. base R or ggplot, but easiest way is with interactions package
library(interactions) ; library(patchwork) # use patchwork to arrange plots
lm(Petal.Length ~ Petal.Width * Species, iris)      %>%  interact_plot(Petal.Width, Species)  +
lm(Petal.Length ~ Petal.Width * Sepal.Width, iris)  %>%  interact_plot(Petal.Width, Sepal.Width)
```



`Petal.Width` interacts with `Species` but not `Sepal.Width` for predicting `Petal.Length`

Interaction summary

Summary for previous `Petal.Length ~ Petal.Width * Species` + other `Sepal` columns:

```
model.iris2 = lm(Petal.Length ~ . + Petal.Width * Species, iris)
summary(model.iris2)
```

```
                            Estimate Std. Error t value Pr(>|t|)
(Intercept)                -0.84034    0.27219  -3.087  0.00243 **
Sepal.Length                0.58888    0.04820  12.218  < 2e-16 ***
Sepal.Width                -0.20027    0.07688  -2.605  0.01016 *
Petal.Width                 0.16640    0.34517   0.482  0.63050
Speciesversicolor           0.51475    0.29040   1.773  0.07845 .
Speciesvirginica            2.31734    0.30982   7.480 7.08e-12 ***
Petal.Width:Speciesversicolor  1.07399    0.38567   2.785  0.00609 **
Petal.Width:Speciesvirginica   0.22406    0.36427   0.615  0.53949
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

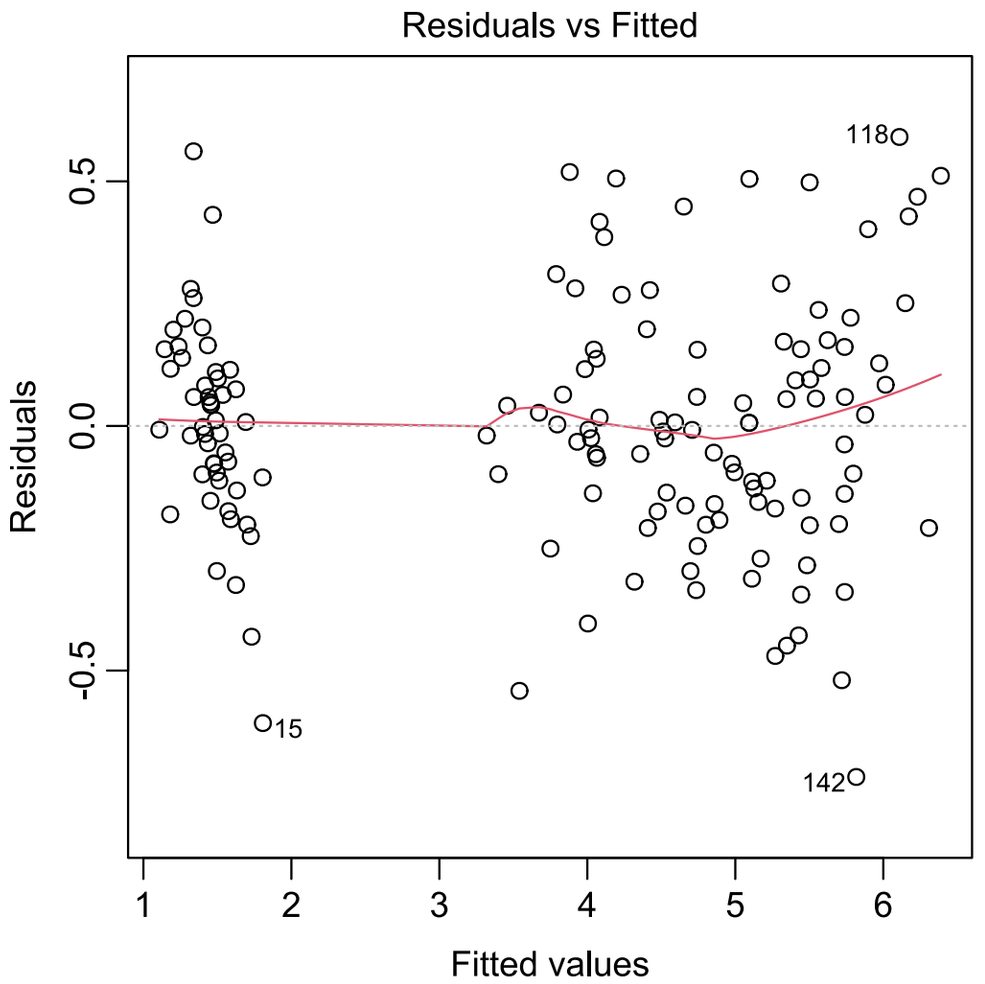
Residual standard error: 0.2507 on 142 degrees of freedom
Multiple R-squared:  0.9808,    Adjusted R-squared:  0.9798
F-statistic:  1036 on 7 and 142 DF,  p-value: < 2.2e-16
```

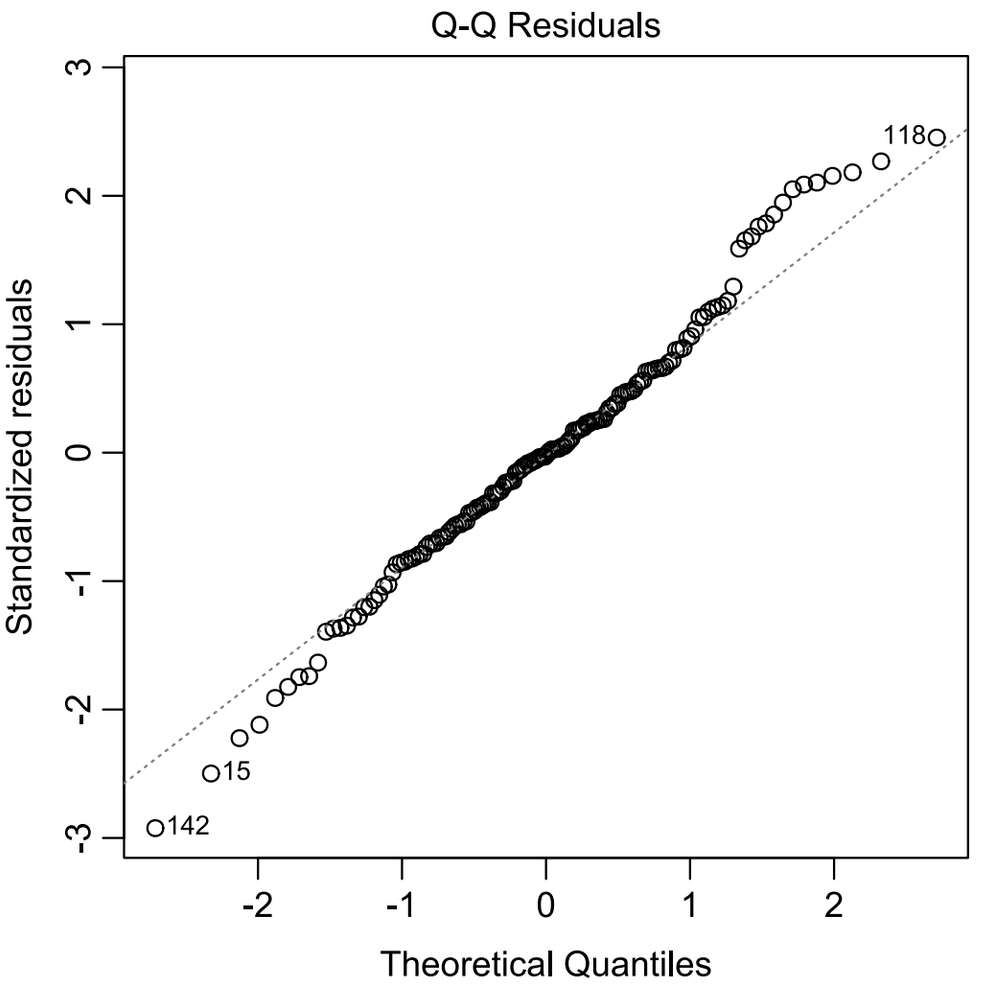
Interpretation: `Species` significantly interacts with `Petal.Width`, specifically versicolor irises have higher `Petal.Width` slope by 1.07 vs setosa ($p = 0.006$).

Note: `Petal.Width` not significant by itself, but it should **not be removed** if its interaction is present, this is called **hierarchical principle** or principle of marginality.

Quick residual check

```
plot(model.iris2, which=1:2)
```





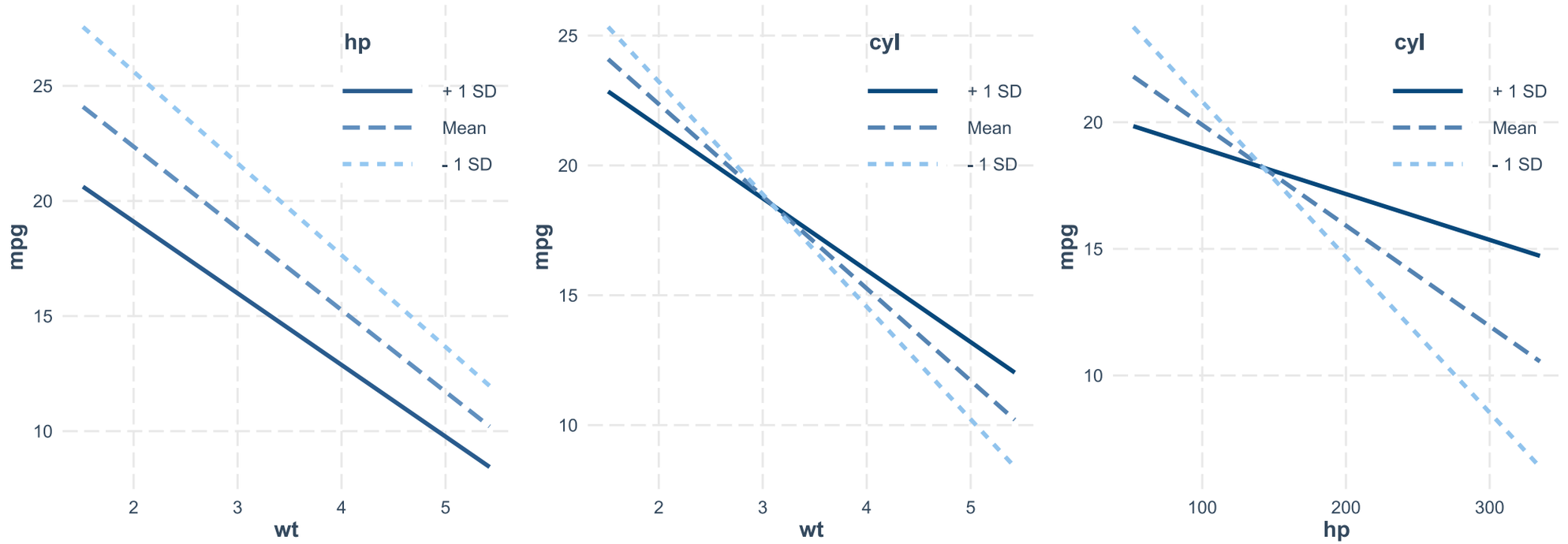
Interaction exercise

Using `mtcars` again, this time fit a model predicting `mpg` using `hp`, `wt`, `cyl`. Check for any significant interactions, fit and interpret them, then check residuals.

Interaction exercise

Using `mtcars` again, this time fit a model predicting `mpg` using `hp`, `wt`, `cyl`. Check for any significant interactions, fit and interpret them, then check residuals.

```
# add all 2-way interactions and check plots
lm(mpg ~ (hp + wt + cyl)^2, mtcars) %>% interact_plot(wt, hp)  +
lm(mpg ~ (hp + wt + cyl)^2, mtcars) %>% interact_plot(wt, cyl) +
lm(mpg ~ (hp + wt + cyl)^2, mtcars) %>% interact_plot(hp, cyl)
```



Seems `cyl` may interact with both `hp` and `wt`, so try adding in those

Interaction exercise (cont.)

```r
model.mpg = lm(mpg ~ cyl * (hp + wt), mtcars)
summary(model.mpg)
```

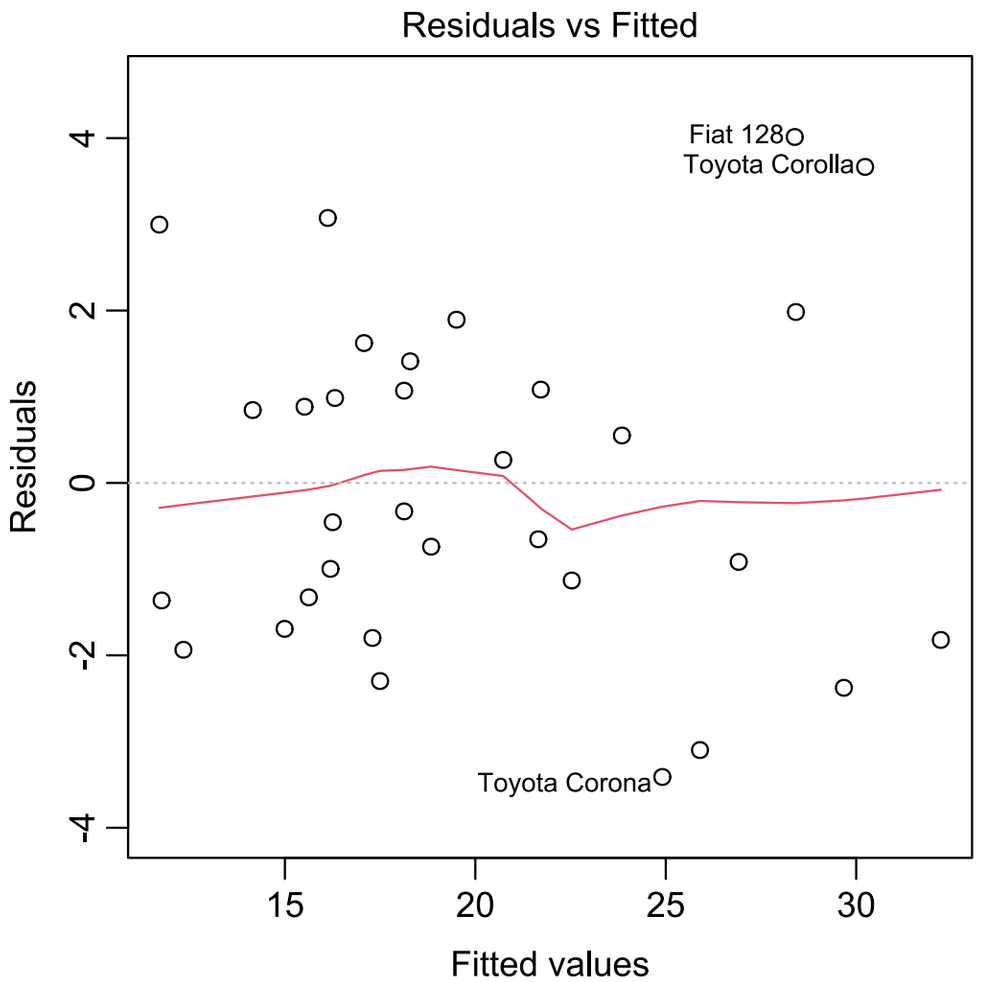
```
Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 59.478155   6.066208   9.805 3.19e-10 ***
cyl         -3.932279   0.979421  -4.015  0.00045 ***
hp          -0.124653   0.053577  -2.327  0.02805 *
wt          -7.386771   2.292623  -3.222  0.00341 **
cyl:hp       0.013537   0.006922   1.956  0.06133 .
cyl:wt       0.629755   0.325337   1.936  0.06385 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

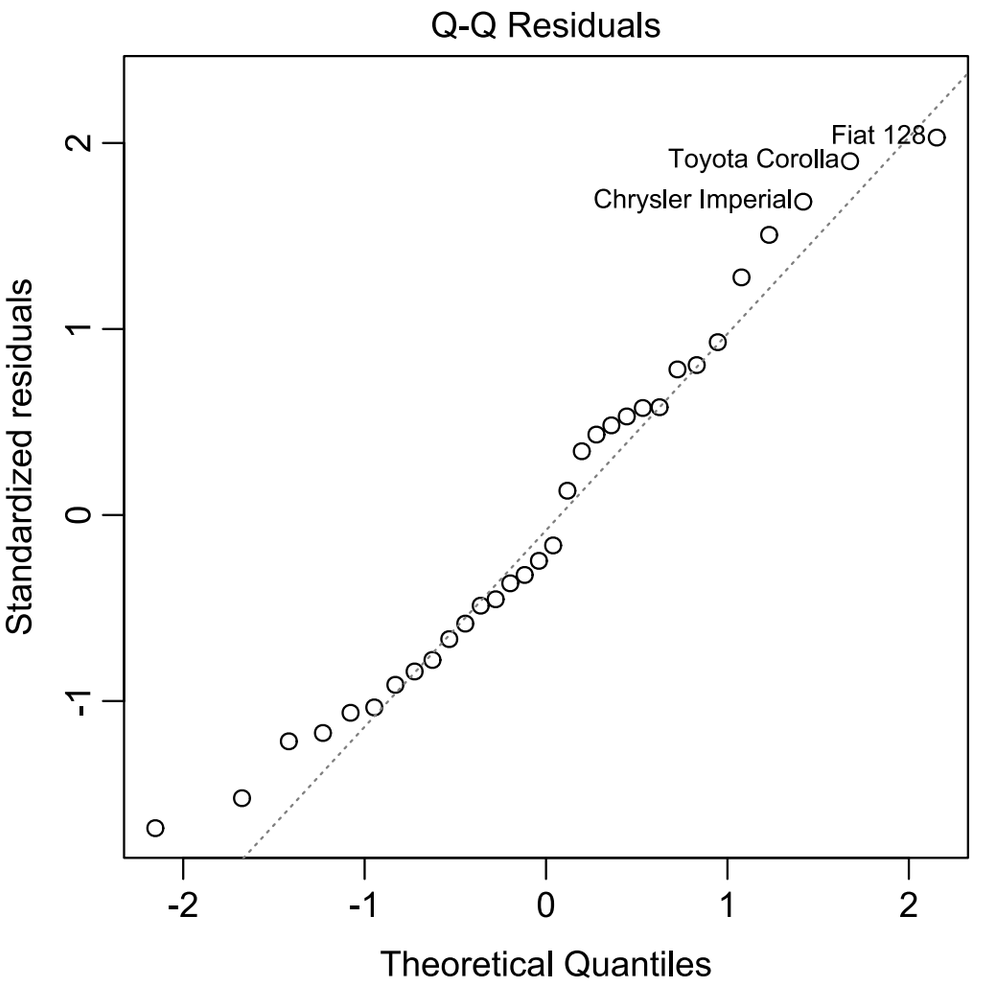
Residual standard error: 2.136 on 26 degrees of freedom
Multiple R-squared:  0.8946,    Adjusted R-squared:  0.8744
F-statistic: 44.16 on 5 and 26 DF,  p-value: 6.826e-12
```

Cylinder seems to weakly interact with horsepower and weight ($p \approx 0.06$ for both), each additional cylinder increases effect on mpg from horsepower by 0.014 and wt by 0.63 on average, holding other vars constant.

Interaction exercise (cont.)

```
plot(model.mpg, which=1:2)
```





Interaction (cont.)

A few additional points about interactions:

- 2-way is most common, but can have 3-way, 4-way, etc… no limit, but the higher the interaction order the weaker/harder-to-fit the effect tends to be
- Hierarchical principle applies to any-way, if you include an n-way effect, all lower order effects for those variables should be in model
- In `R`, different ways to add interactions:
  - `x1:x2` adds only the specific interaction
  - `x1*x2` adds interaction plus all lower-order effects
  - `( )^2` shortcut for adding 2-way interactions between several variables

Interaction (cont.)

A few additional points about interactions:

- 2-way is most common, but can have 3-way, 4-way, etc… no limit, but the higher the interaction order the weaker/harder-to-fit the effect tends to be
- Hierarchical principle applies to any-way, if you include an n-way effect, all lower order effects for those variables should be in model
- In `R`, different ways to add interactions:
  - `x1:x2` adds only the specific interaction
  - `x1*x2` adds interaction plus all lower-order effects
  - `( )^2` shortcut for adding 2-way interactions between several variables

- Equivalent to adding **the product of each predictor's coefficient(s)** to model, e.g. suppose we have `y ~ (x1) * (x2)`
  - If `x1` is numeric, it contributes a single coefficient β_1 to the product with `x2`
  - If `x1` is categorical with n levels, it contributes $n-1$ coefficients $\beta_1, \ldots, \beta_{n-1}$ to the product with `x2`, and **each combination of `x1`'s and `x2`'s coefficients appears**!

Interaction advanced example

Suppose we are predicting flipper length again for penguins with the following lm model.

```
model.penguins2 = lm(flipper_len ~ (body_mass + species) * (bill_len + sex), drop_na(penguins))
```

Body mass, bill length are numeric. Species is 3-level categorical, and sex is 2-level. Each variable "multiplied" in the `lm` formula adds the interaction.

Questions:

- How many variables are interacted, and what order of interactions are these?
- How many coefficients will appear in the summary output?
- The data frame has 333 complete rows. How many residual degrees of freedom are there?

Interaction advanced example answers

Suppose we are predicting flipper length again for penguins with the following lm model.

```
model.penguins2 = lm(flipper_len ~ (body_mass + species) * (bill_len + sex), drop_na(penguins))
```

Body mass, bill length are numeric. Species is 3-level categorical, and sex is 2-level. Each variable "multiplied" in the `lm` formula adds the interaction.

- There are 4 different 2-way interactions added: `body_mass:bill_len`, `body_mass:sex`, `species:bill_len`, `species:sex`
- 12 total coefficients: 1 intercept +
  - `body_mass` and `bill_len` each adds $1 + 1 = 2$ coefficients
  - `sex` adds $2 - 1 = 1$ coefficient, and `species` adds $3 - 1 = 2$ coefficients
  - `body_mass:bill_len` adds $1 \times 1 = 1$ coefficient
  - `body_mass:sex` also adds $1 \times 1 = 1$ coefficient
  - `species:bill_len` adds $2 \times 1 = 2$ coefficients
  - `species:sex` also adds $2 \times 1 = 2$ coefficients
- There will be $333 - 12 = 321$ residual degrees of freedom

`lm` summary

```r
summary(model.penguins2)  # lm summary of previous model
```

```
                          Estimate Std. Error t value Pr(>|t|)
(Intercept)               1.861e+02  2.623e+01   7.095 8.31e-12 ***
body_mass                -2.107e-03  6.997e-03  -0.301   0.7635
speciesChinstrap          9.166e+00  1.436e+01   0.638   0.5238
speciesGentoo             1.602e+01  1.754e+01   0.913   0.3617
bill_len                 -3.598e-01  6.397e-01  -0.562   0.5742
sexmale                  -2.144e+00  7.901e+00  -0.271   0.7862
body_mass:bill_len        1.767e-04  1.665e-04   1.062   0.2892
body_mass:sexmale         6.466e-04  2.083e-03   0.310   0.7564
speciesChinstrap:bill_len -1.807e-01  3.300e-01  -0.548   0.5844
speciesGentoo:bill_len    -1.904e-02  4.093e-01  -0.047   0.9629
speciesChinstrap:sexmale   4.568e+00  2.190e+00   2.086   0.0378 *
speciesGentoo:sexmale      3.061e-01  3.064e+00   0.100   0.9205
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.196 on 321 degrees of freedom
Multiple R-squared:  0.8671,    Adjusted R-squared:  0.8626
F-statistic: 190.4 on 11 and 321 DF,  p-value: < 2.2e-16
```

Question: suppose we sample a new female Gentoo penguin with bodymass 5000g and bill length 40mm. What would this model predict for the flipper length in mm?

`lm` summary

```
summary(model.penguins2)  # lm summary of previous model
```

```
                          Estimate Std. Error t value Pr(>|t|)
(Intercept)                1.861e+02  2.623e+01   7.095 8.31e-12 ***
body_mass                 -2.107e-03  6.997e-03  -0.301   0.7635
speciesChinstrap           9.166e+00  1.436e+01   0.638   0.5238
speciesGentoo              1.602e+01  1.754e+01   0.913   0.3617
bill_len                  -3.598e-01  6.397e-01  -0.562   0.5742
sexmale                   -2.144e+00  7.901e+00  -0.271   0.7862
body_mass:bill_len         1.767e-04  1.665e-04   1.062   0.2892
body_mass:sexmale          6.466e-04  2.083e-03   0.310   0.7564
speciesChinstrap:bill_len -1.807e-01  3.300e-01  -0.548   0.5844
speciesGentoo:bill_len    -1.904e-02  4.093e-01  -0.047   0.9629
speciesChinstrap:sexmale   4.568e+00  2.190e+00   2.086   0.0378 *
speciesGentoo:sexmale      3.061e-01  3.064e+00   0.100   0.9205
```

```
# manually, note any "sexmale" rows do NOT apply
186.1 + -0.002107*5000 + 16 + -0.3598*40 + 0.0001767*5000*40 + -0.01904*1*40
```

```
[1] 211.7514
```

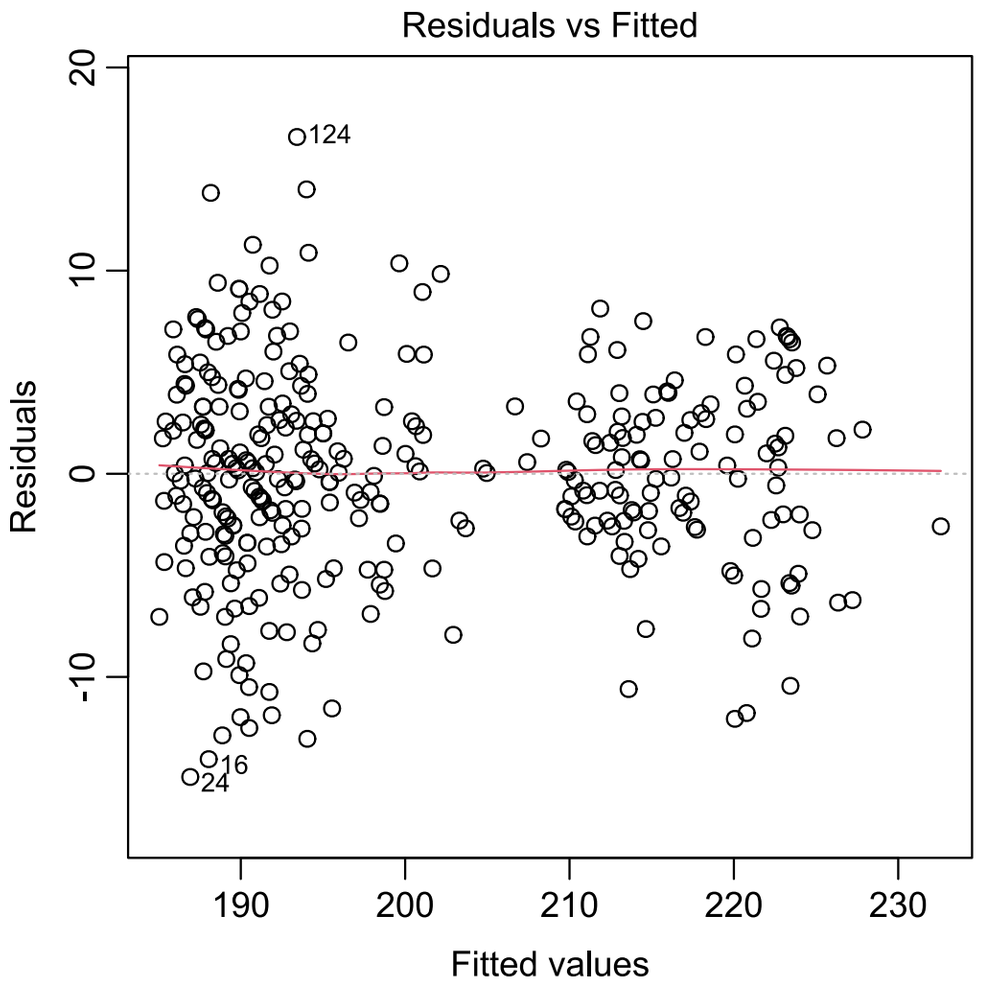
```
# with predict() function
predict(model.penguins2, tibble(body_mass=5000, bill_len=40, sex="female", species="Gentoo"))
```

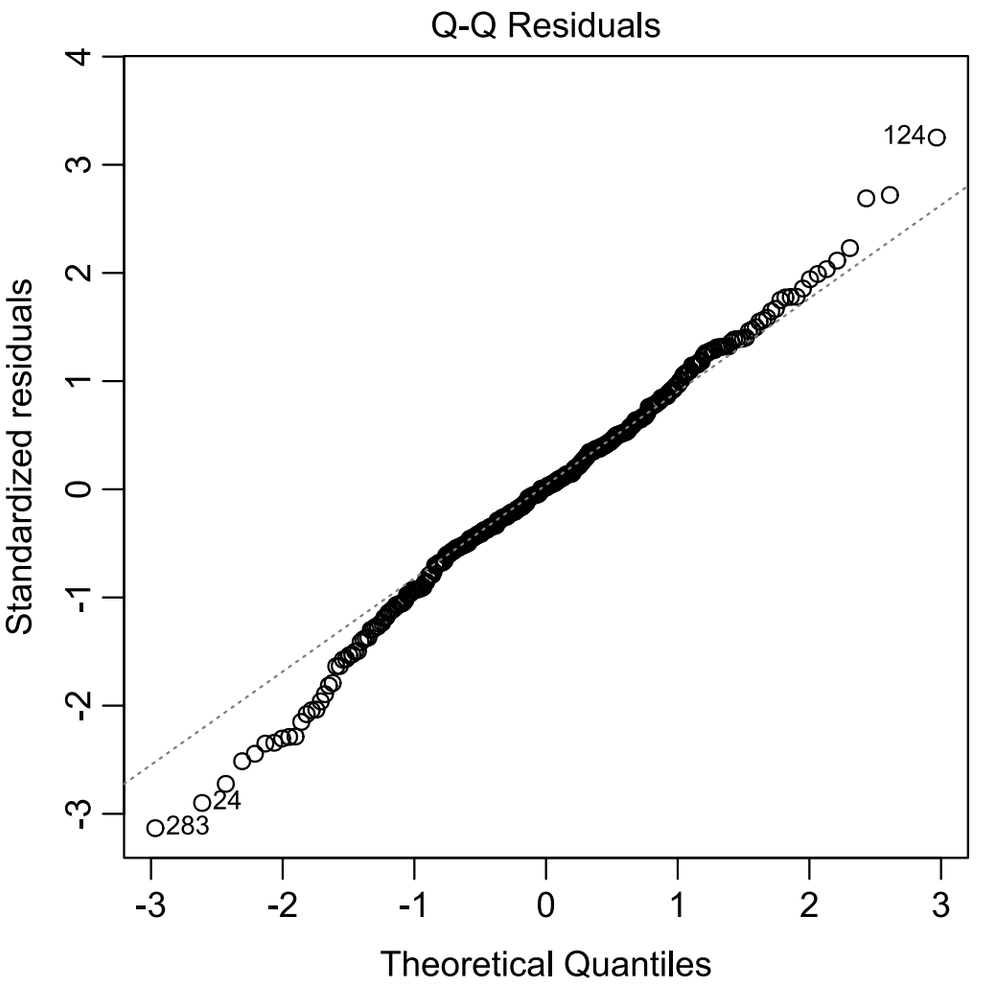
```
[1] 211.775
```

Female Gentoo penguins with bodymass 5000g and bill length 40mm predicted to have 212mm flipper length on average.

Quick residual check

```
plot(model.penguins2, which=1:2)
```

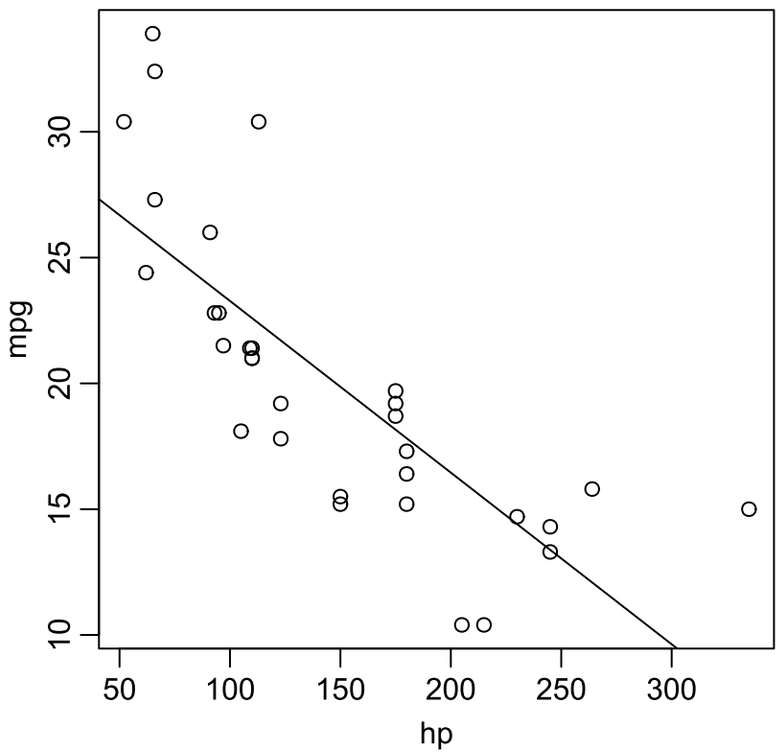


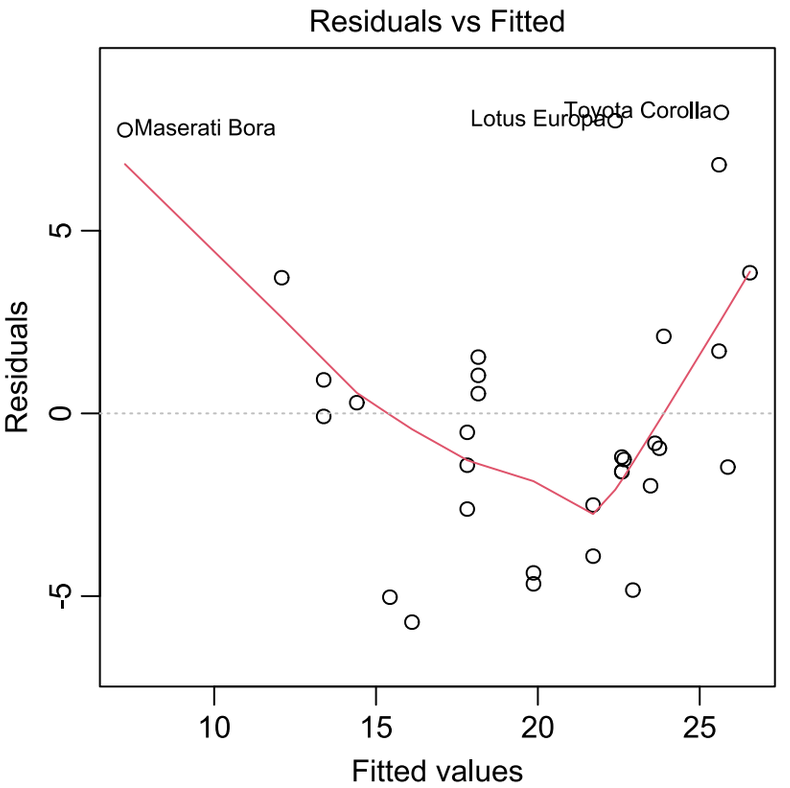


Nonlinear terms

Sometimes, higher order effects of a predictor may be needed, these can also be easily added in R. Example: using `mtcars` again, suppose we only wish to predict `mpg` with `hp`. Plot:

```
lm.hp = lm(mpg ~ hp, mtcars)  ;  coefs = coef(lm.hp)
with(mtcars, plot(hp, mpg))   ;  abline(coefs[1],coefs[2])  ;  plot(lm.hp,1)
```





There's clear non-linearity present, so residual mean-0 assumption is violated. This can be fixed in other ways (e.g. adding interactions) but can also try adding a squared `hp` term

Nonlinear terms `hp` example

For squared term, add `+ I(hp^2)` where `I()` is the "AsIs" function that disables `R`'s formula interpreter and leaves it as-is as a transformation function

```
lm.hp2 = lm(mpg ~ hp + I(hp^2), mtcars)
summary(lm.hp2)
```

```
Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  4.041e+01  2.741e+00  14.744 5.23e-15 ***
hp          -2.133e-01  3.488e-02  -6.115 1.16e-06 ***
I(hp^2)      4.208e-04  9.844e-05   4.275 0.000189 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.077 on 29 degrees of freedom
Multiple R-squared:  0.7561,    Adjusted R-squared:  0.7393
F-statistic: 44.95 on 2 and 29 DF,  p-value: 1.301e-09
```

Nonlinear terms `hp` example

For squared term, add `+ I(hp^2)` where `I()` is the "AsIs" function that disables R's formula interpreter and leaves it as-is as a transformation function

```
lm.hp2 = lm(mpg ~ hp + I(hp^2), mtcars)
summary(lm.hp2)
```

```
Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  4.041e+01  2.741e+00  14.744 5.23e-15 ***
hp          -2.133e-01  3.488e-02  -6.115 1.16e-06 ***
I(hp^2)      4.208e-04  9.844e-05   4.275 0.000189 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

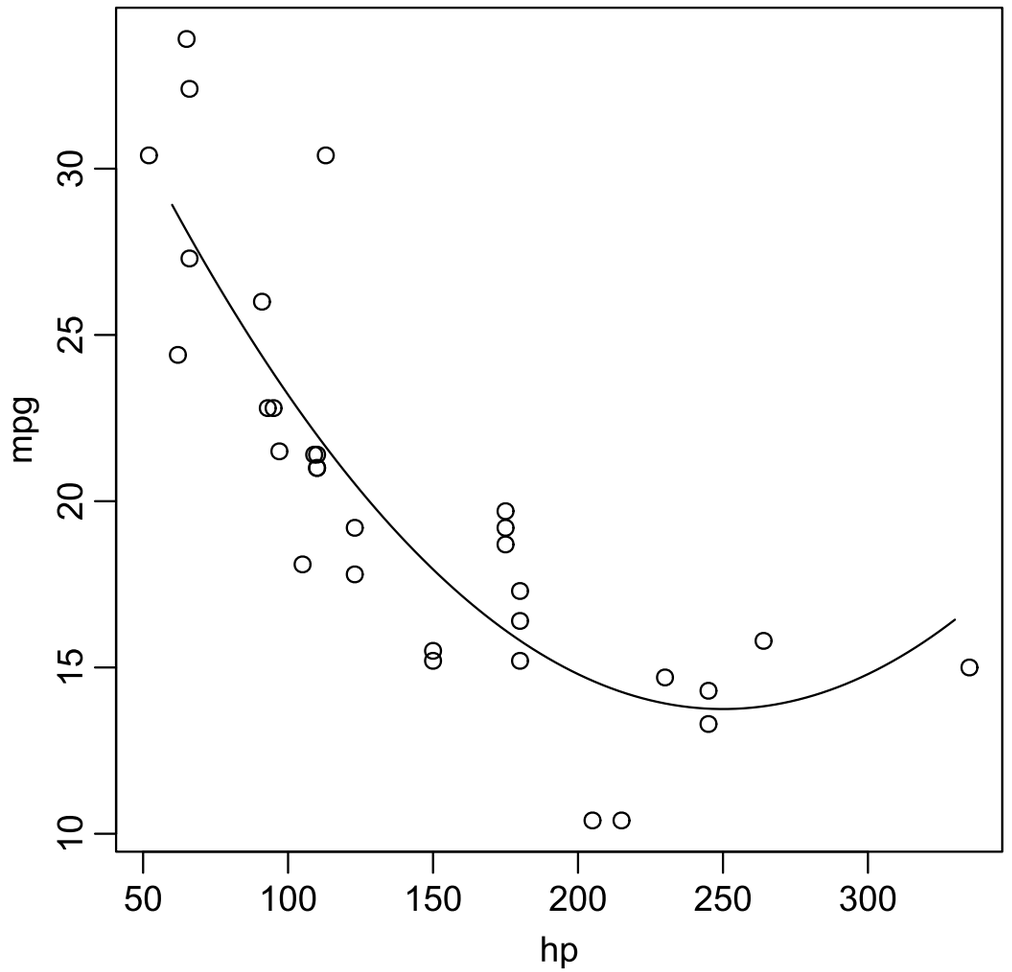
Residual standard error: 3.077 on 29 degrees of freedom
Multiple R-squared:  0.7561,    Adjusted R-squared:  0.7393
F-statistic: 44.95 on 2 and 29 DF,  p-value: 1.301e-09
```

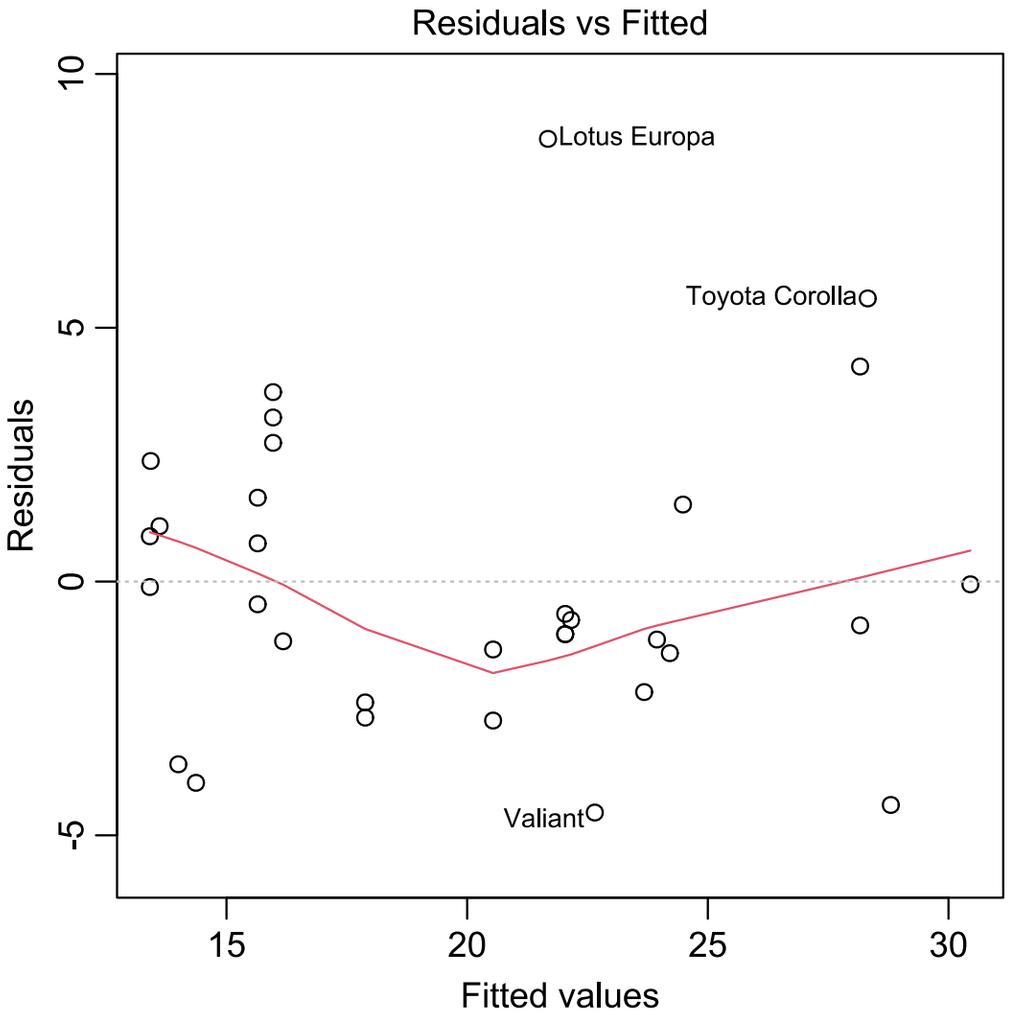
Squared `hp` term is quite significant here ($p \approx 10^{-4}$) and dramatically improves model performance (R^2 improved from 0.602 to 0.756)

Note: hierarchical principle applies here too, if there's a higher order (e.g. squared) term, should also keep all lower order terms

Nonlinear terms hp example plots

```
with(mtcars,plot(hp,mpg)) ; plot(\(x)40-.21*x+.00042*x^2,add=T,from=60,to=330) ; plot(lm.hp2,1)
```





Exercise

Extend the previous example to include a 3rd order `hp` term, feel free to add other predictors as well. Which orders of `hp` are significant for predicting `mpg`?

Exercise

Extend the previous example to include a 3rd order `hp` term, feel free to add other predictors as well. Which orders of `hp` are significant for predicting `mpg`?

```
# I arbitrarily pick a few other predictors to include
model.mpg = lm(mpg ~ hp + I(hp^2) + I(hp^3) + wt + cyl, mtcars)  ;  summary(model.mpg)
```

```
Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  4.844e+01  4.538e+00  10.674 5.36e-11 ***
hp          -2.439e-01  9.152e-02  -2.665 0.013059 *
I(hp^2)      1.105e-03  5.153e-04   2.144 0.041518 *
I(hp^3)     -1.657e-06  9.142e-07  -1.812 0.081484 .
wt          -3.056e+00  7.083e-01  -4.315 0.000205 ***
cyl         -4.221e-01  5.925e-01  -0.712 0.482604
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.319 on 26 degrees of freedom
Multiple R-squared:  0.8759,    Adjusted R-squared:  0.852
F-statistic:  36.7 on 5 and 26 DF,  p-value: 5.585e-11
```

With `wt` and `cyl` included, seems like `hp` significant only up to second order. `wt` continues to be very important predictor.

Other nonlinear situations

Log transforms also very common in data science and may be applied to predictors and/or response depending on situation. Generally a variable should ONLY be logged if it:

1. Must be positive,
2. Is right-skewed (or spans many orders of magnitude),
3. It's connected to some theoretical model equation, e.g.:
    - If $y = ab^x$ then $\log(y) = \log(a) + \log(b) \cdot x$, called **exponential** relation
    - If $y = ax^b$ then $\log(y) = \log(a) + b \cdot \log(x)$, called **power law**

If log is necessary but not performed, will often see problems in residual plots.

`trees` exercise

Exercise: `trees` is a default dataset with `Girth` (diameter in inches measured 4.5ft above ground), `Height` (feet), and `Volume` of produced lumber (ft^3) data for 31 black cherry trees.

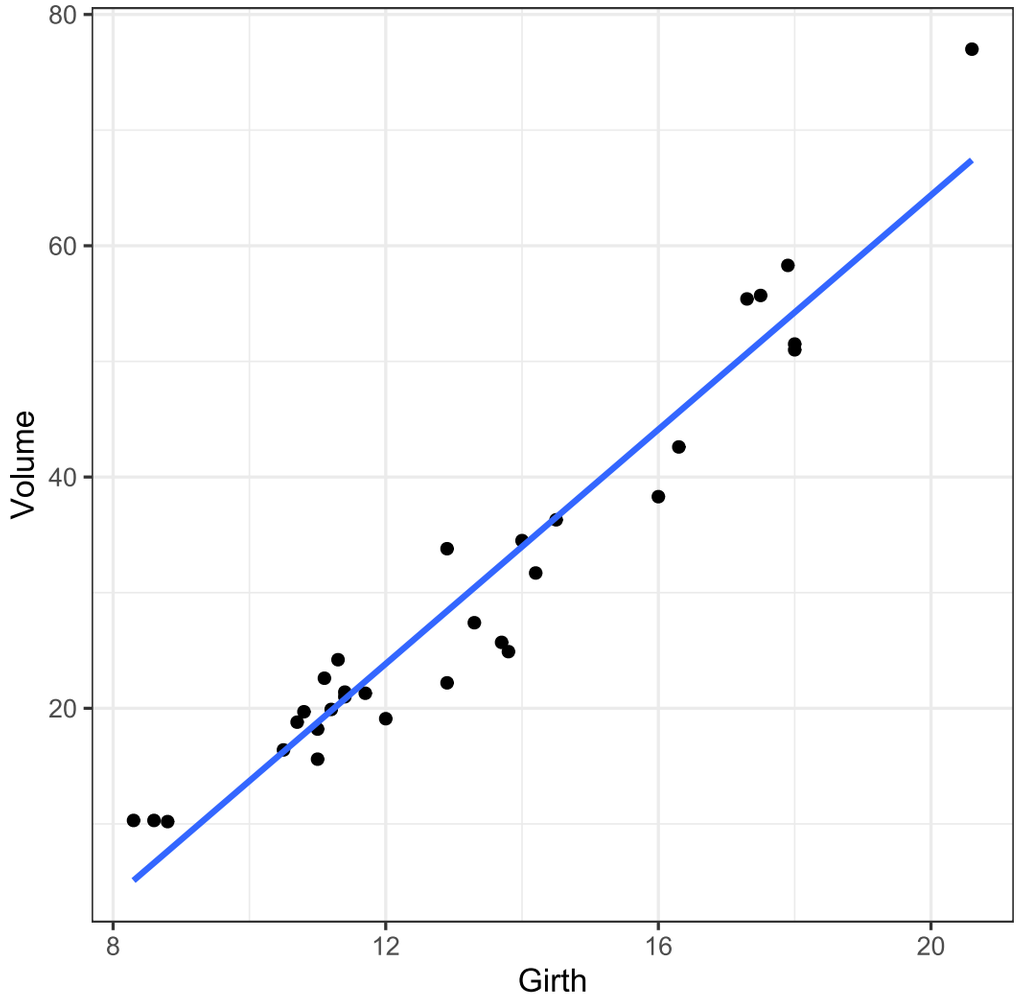
```
glimpse(trees)
```

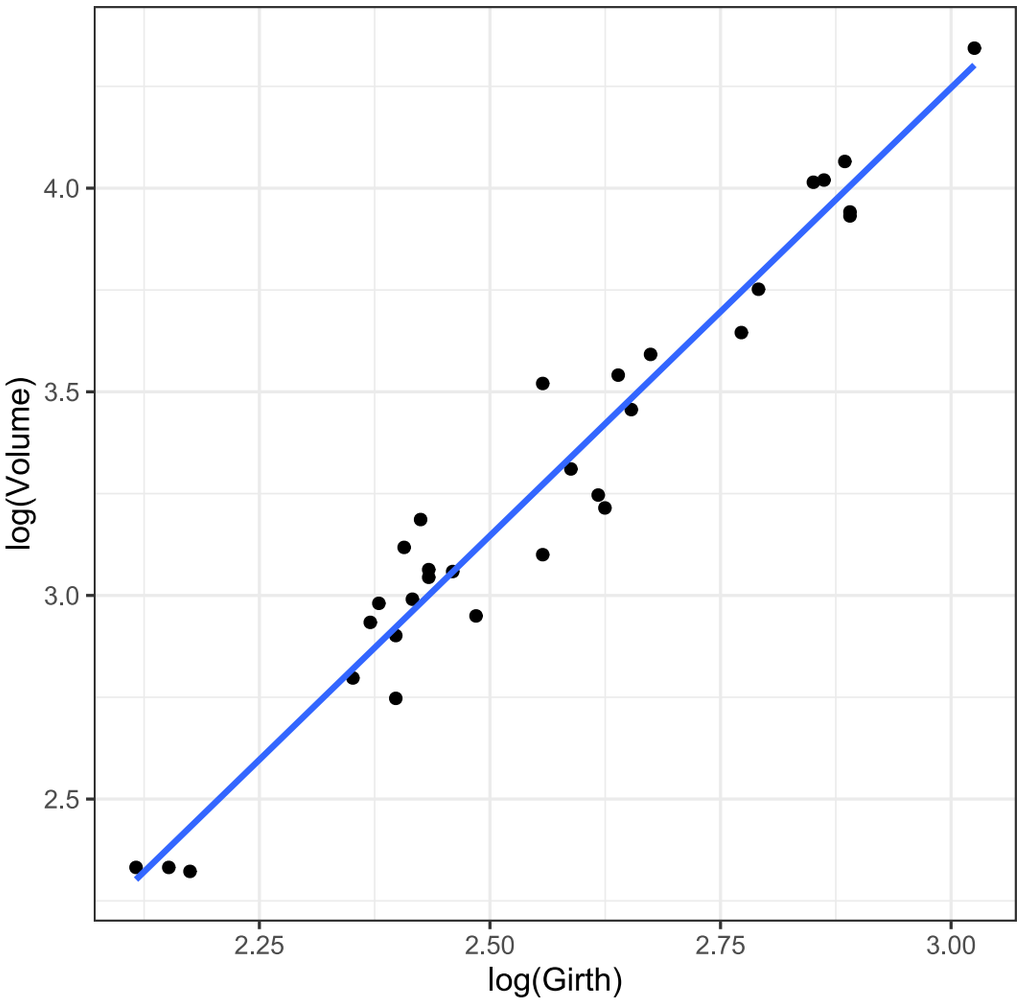
```
Rows: 31
Columns: 3
$ Girth  <dbl> 8.3, 8.6, 8.8, 10.5, 10.7, 10.8, 11.0, 11.0, 11.1, 11.2, 11.3, 11.4, 11.4, 11.7, 12…
$ Height <dbl> 70, 65, 63, 72, 81, 83, 66, 75, 80, 75, 79, 76, 76, 69, 75, 74, 85, 86, 71, 64, 78,…
$ Volume <dbl> 10.3, 10.3, 10.2, 16.4, 18.8, 19.7, 15.6, 18.2, 22.6, 19.9, 24.2, 21.0, 21.4, 21.3,…
```

1. Plot volume of lumber (Y) vs girth of trees (X)
2. Since we expect volume to be proportional to some power of girth, fit a power law model and plot the result. Note `log()` can be directly used in `lm` formula expressions.
3. How predictive is the model?
4. Give point and interval estimates for a, b.
5. Remember to check the residuals.

`trees` exercise answers

```
# raw and log-log transformed data, there is some evidence of nonzero-mean in first plot
ggplot(trees,aes(Girth,Volume)) + geom_point() + geom_smooth(method=lm,se=F) +
ggplot(trees,aes(log(Girth),log(Volume))) + geom_point() + geom_smooth(method=lm,se=F)
```





`trees` exercise answers (cont.)

```
model.trees = lm(log(Volume) ~ log(Girth), trees)  ;  summary(model.trees)
```

```
Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -2.35332    0.23066  -10.20 4.18e-11 ***
log(Girth)   2.19997    0.08983   24.49  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.115 on 29 degrees of freedom
Multiple R-squared:  0.9539,    Adjusted R-squared:  0.9523
F-statistic: 599.7 on 1 and 29 DF,  p-value: < 2.2e-16
```

```
confint(model.trees)
```

```
                2.5 %    97.5 %
(Intercept) -2.825083 -1.881566
log(Girth)   2.016238  2.383702
```

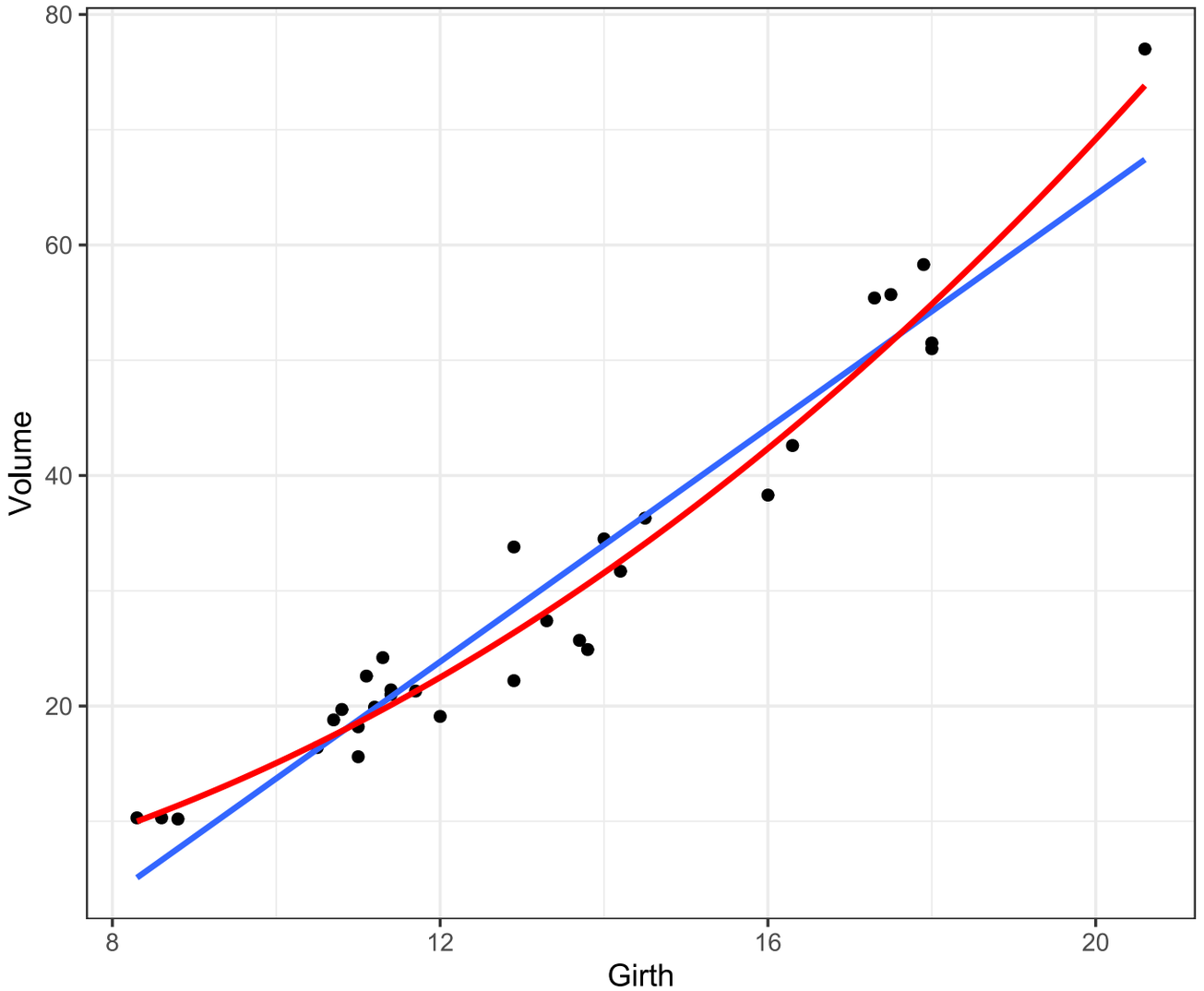
```
exp(confint(model.trees)[1,])
```

```
     2.5 %     97.5 %
0.05930371 0.15235127
```

The model estimates $\hat{b} = 2.2$ and performs very well, explaining 95% of the variations in volume. 95% confidence intervals for a and b are $(0.06, 0.15)$ and $(2.0, 2.4)$.

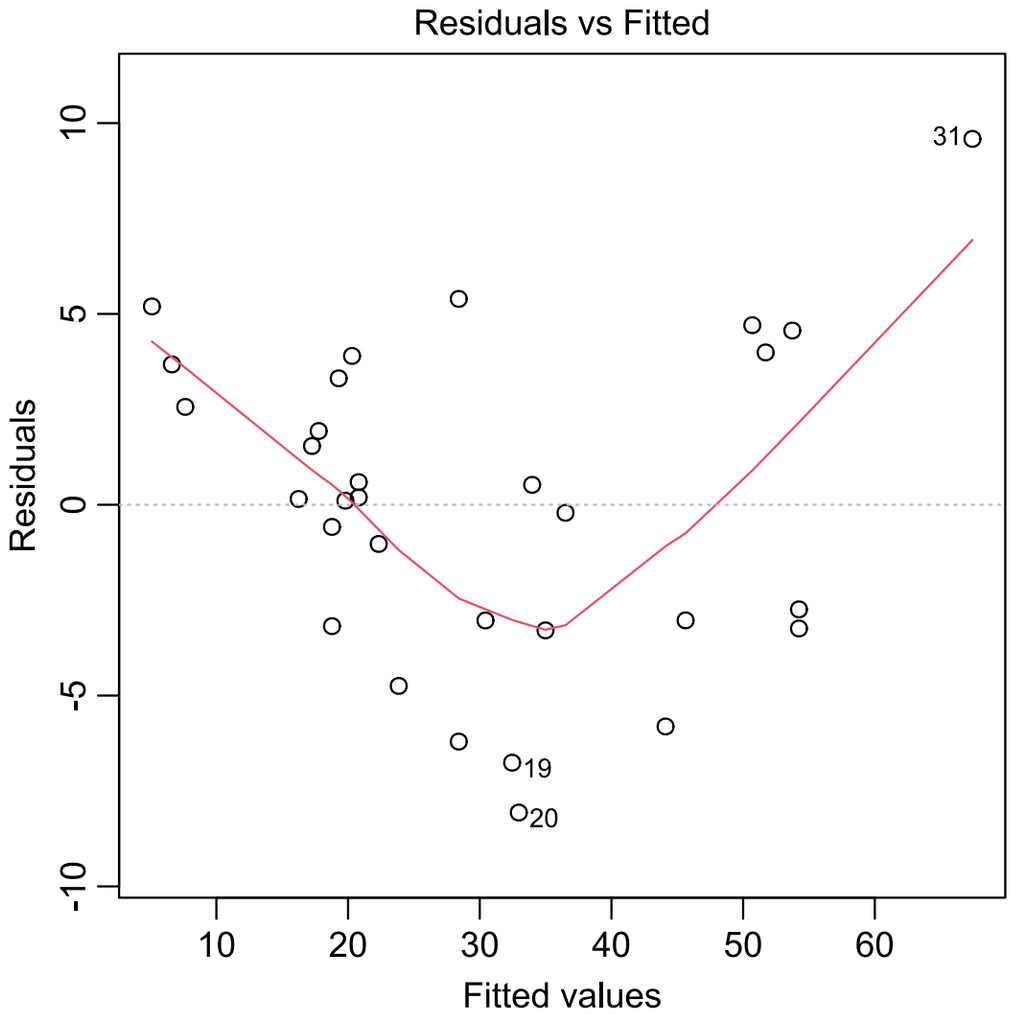
`trees` exercise answers (cont.)

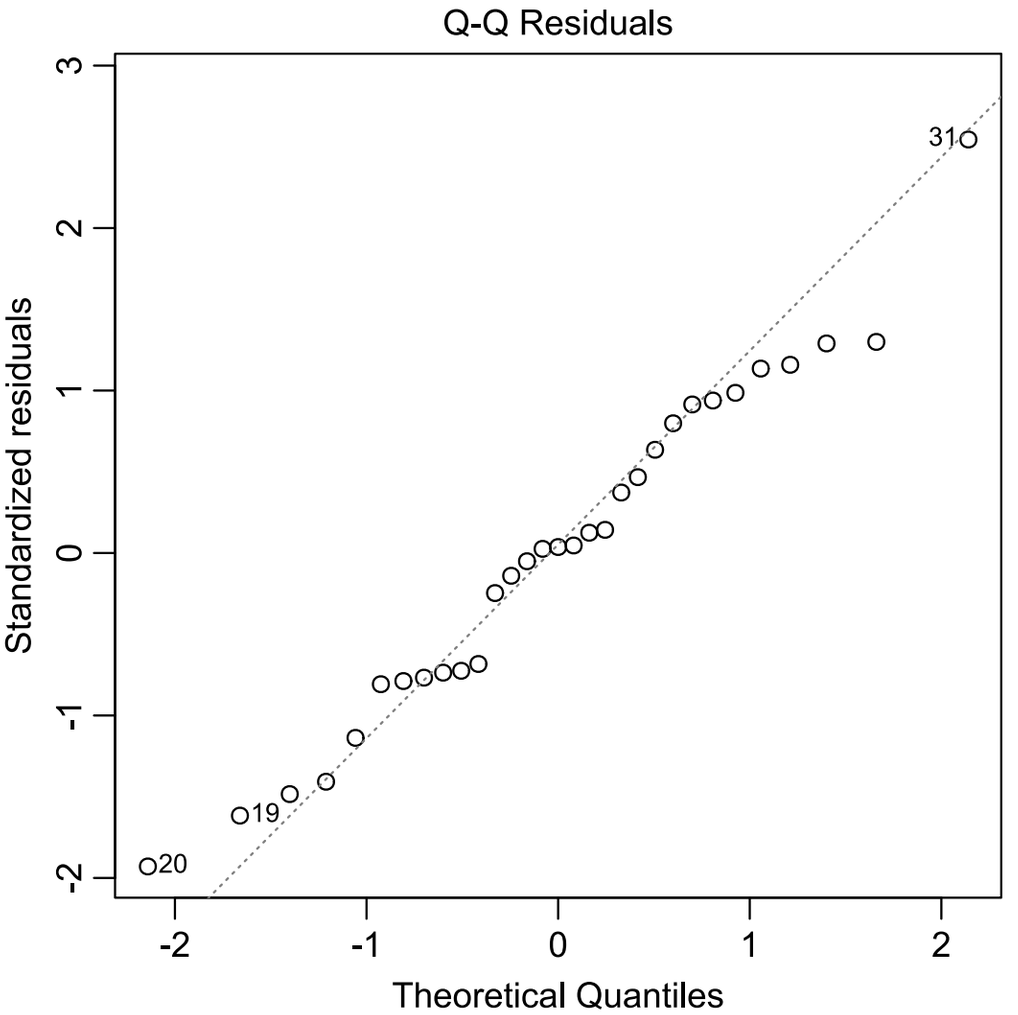
```
# replotting power law with original data, compared with straight line
ggplot(trees,aes(Girth,Volume)) + geom_point() + geom_smooth(method=lm,se=F) +
  geom_function(fun=\(x)0.095*x^2.2, color="red", linewidth=1)
```



`trees` exercise answers (cont.)

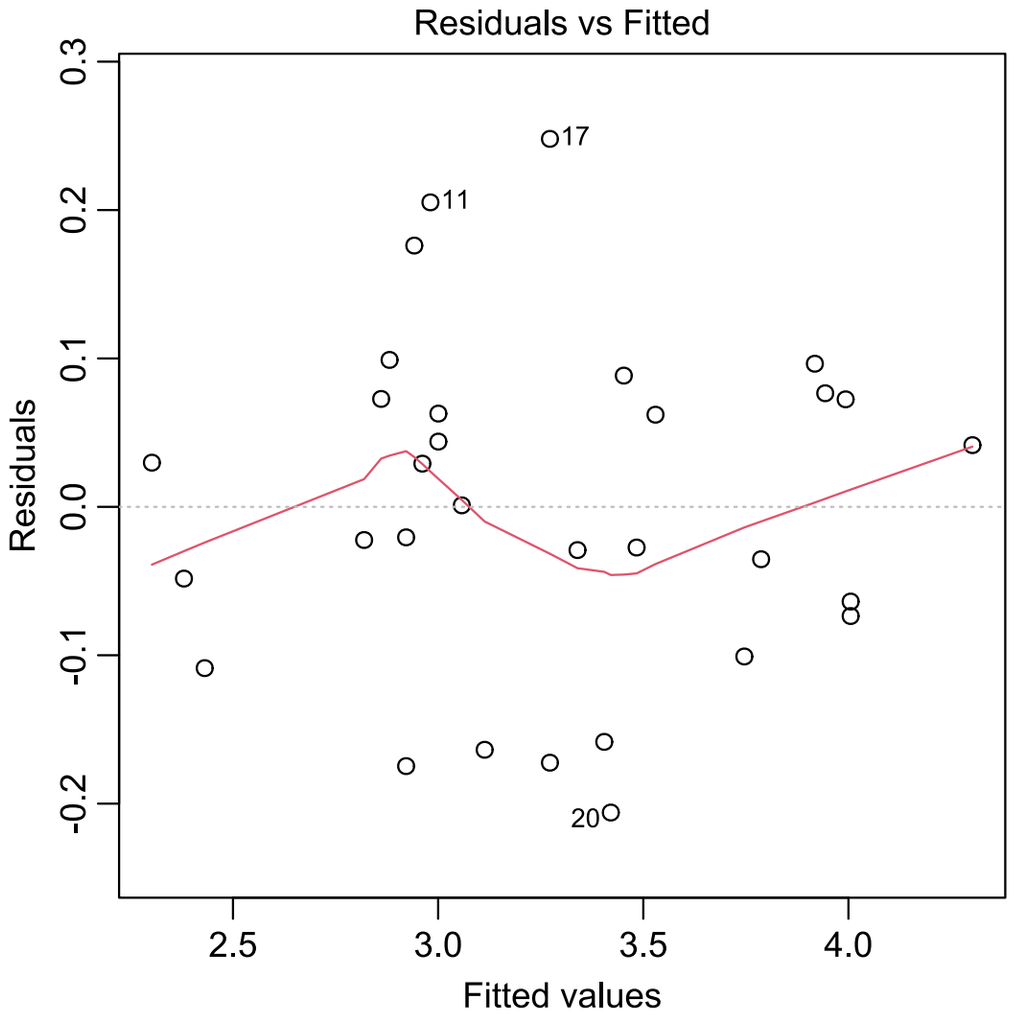
```
plot(lm(Volume~Girth,trees), which=1:2)  # without power law, visible nonlinearity
```

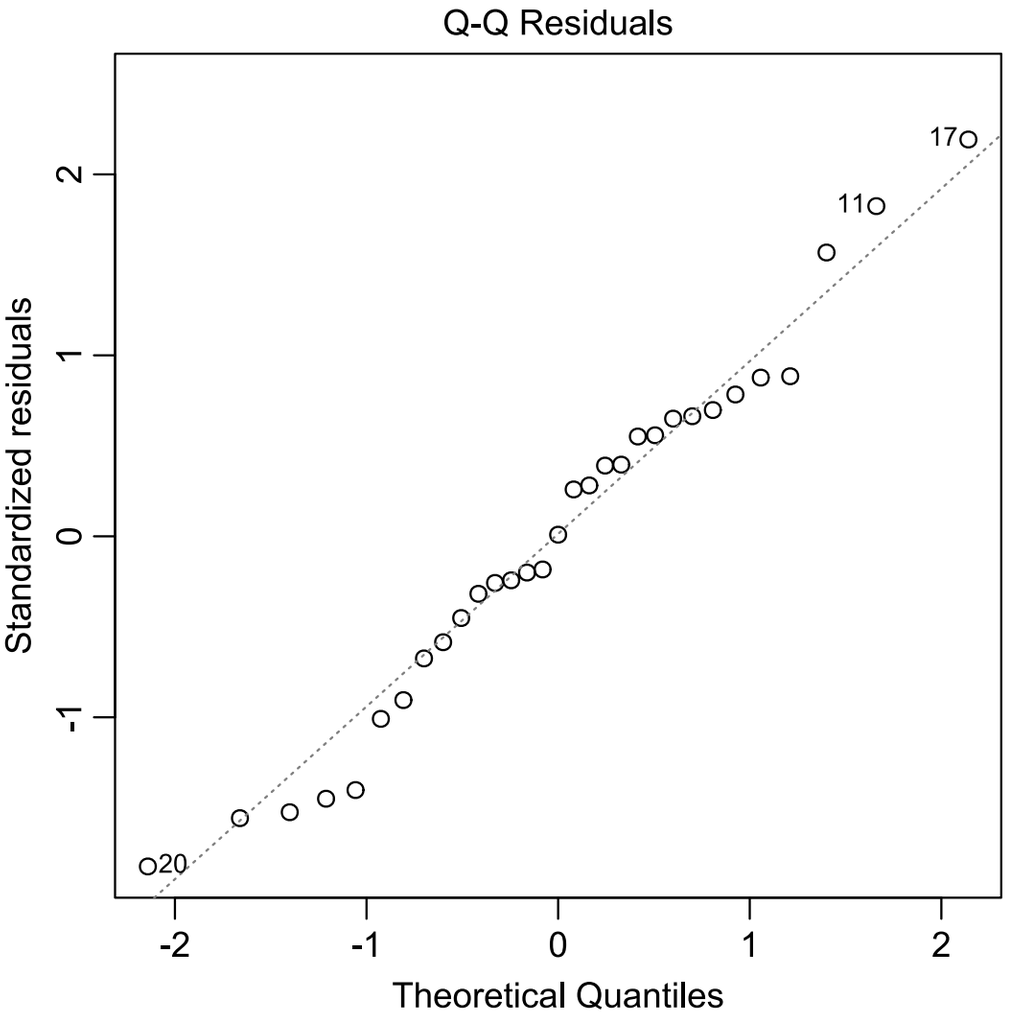




`trees` exercise answers (cont.)

```
plot(model.trees, which=1:2)  # with power law, residuals look much better
```





(more notes added soon)